

Cooperative State University Baden-Württemberg  
Center for Advanced Studies

**Master thesis**

**Improving and Implementing a Dynamic,  
Bézier-based Undergirding Bonding Curve Market  
Maker for Application in Blockchain-based  
Cryptoeconomic Systems**

**of the course of studies in business informatics**

|                    |                             |
|--------------------|-----------------------------|
| Author:            | Daniel Kirste               |
| Matr. number:      | 3678893                     |
| Course:            | CAS-WMWI21-W                |
| Company:           | Robert Bosch GmbH           |
| First evaluator:   | Prof. Dr. Andreas Mitschele |
| Second evaluator:  | Prof. Dr. Sebastian Richter |
| Processing period: | 01.04.2022 – 01.08.2022     |
| Submission date:   | 01.08.2022                  |

# Declaration of Oath

I hereby confirm that I have written this thesis with the title:

*“Improving and Implementing a Dynamic, Bézier-based Undergirding Bonding Curve Market Maker for Application in Blockchain-based Cryptoeconomic Systems”*

on my own and that I have not used any other media or materials than the ones referred to in this thesis. Any concepts or quotations applicable to these sources are clearly attributed to them.

Place, Date

Daniel Kirste

# Non-disclosure Notice

This Master's thesis contains internal and confidential information from Robert Bosch GmbH.

Third parties - with the exception of the thesis supervisor and authorized examining persons - may not access this work without the express permission of the aforementioned company and author.

The information contained in this document may not be disclosed publicly, unless otherwise expressly approved by the company and author.

# Contents

|                                                                   |             |
|-------------------------------------------------------------------|-------------|
| <b>List of Figures</b>                                            | <b>v</b>    |
| <b>List of Tables</b>                                             | <b>vi</b>   |
| <b>Acronyms</b>                                                   | <b>vii</b>  |
| <b>Abstract</b>                                                   | <b>viii</b> |
| <b>1 Introduction</b>                                             | <b>1</b>    |
| 1.1 Problems . . . . .                                            | 2           |
| 1.2 Relevance . . . . .                                           | 3           |
| 1.3 Contributions . . . . .                                       | 4           |
| <b>2 Literature Review</b>                                        | <b>6</b>    |
| 2.1 Cosmos Ecosystem . . . . .                                    | 6           |
| 2.1.1 Cosmos Architecture . . . . .                               | 7           |
| 2.1.2 Cosmos SDK . . . . .                                        | 8           |
| 2.1.3 Ignite CLI . . . . .                                        | 8           |
| 2.2 Automated Market Maker . . . . .                              | 8           |
| 2.2.1 Actors on AMM-based DEXs . . . . .                          | 9           |
| 2.2.2 Fundamental Economics of AMMs . . . . .                     | 10          |
| 2.2.3 Constant Function Market Makers . . . . .                   | 11          |
| 2.3 Bonding Curves . . . . .                                      | 12          |
| 2.3.1 Augmented Bonding Curves . . . . .                          | 14          |
| 2.3.2 Risk-adjusted Bonding Curves . . . . .                      | 14          |
| 2.4 Undergirding Bonding Curve . . . . .                          | 15          |
| 2.4.1 General Mechanisms of Undergirding Bonding Curves . . . . . | 19          |
| 2.4.2 Bézier Curves . . . . .                                     | 21          |
| 2.4.3 Basic Equations for Bézier Curves . . . . .                 | 23          |
| 2.4.4 Curvature Conditions . . . . .                              | 25          |
| 2.4.5 Fitting Two Segments . . . . .                              | 28          |
| 2.4.6 Initializing the UBC . . . . .                              | 30          |
| 2.4.7 Undergirding Actions . . . . .                              | 36          |
| 2.4.8 Buy and Sell Actions . . . . .                              | 37          |

|                   |                                               |           |
|-------------------|-----------------------------------------------|-----------|
| <b>3</b>          | <b>Research Method</b>                        | <b>41</b> |
| <b>4</b>          | <b>Artefact Description</b>                   | <b>44</b> |
| 4.1               | Cycle Zero - Initial Considerations . . . . . | 44        |
| 4.1.1             | Fitting the Curve . . . . .                   | 45        |
| 4.2               | Cycle One . . . . .                           | 46        |
| 4.3               | Cycle Two . . . . .                           | 48        |
| 4.4               | Cycle Three - Fitting algorithm . . . . .     | 50        |
| 4.4.1             | Initializing the Curve . . . . .              | 51        |
| 4.4.2             | Undergirding . . . . .                        | 52        |
| 4.4.3             | Shifting up P2 . . . . .                      | 54        |
| 4.4.4             | Flat region . . . . .                         | 56        |
| 4.4.5             | Quadratic Segment . . . . .                   | 59        |
| 4.4.6             | Fitting Algorithm Evaluation . . . . .        | 61        |
| 4.5               | Cosmos Implementation . . . . .               | 62        |
| 4.5.1             | Basic Structure of the Prototype . . . . .    | 62        |
| 4.5.2             | Ubc Object . . . . .                          | 63        |
| 4.5.3             | Flat Segment . . . . .                        | 64        |
| 4.5.4             | Segment . . . . .                             | 64        |
| 4.5.5             | Quadratic Segment . . . . .                   | 64        |
| 4.5.6             | Bank Integration . . . . .                    | 65        |
| 4.5.7             | Messages . . . . .                            | 66        |
| 4.5.8             | Decimal Representation . . . . .              | 68        |
| <b>5</b>          | <b>Evaluation</b>                             | <b>69</b> |
| 5.1               | Functional requirements . . . . .             | 69        |
| 5.2               | System requirements . . . . .                 | 71        |
| <b>6</b>          | <b>Discussion</b>                             | <b>77</b> |
| 6.1               | Curve shape fitting algorithm . . . . .       | 77        |
| 6.2               | UBC-MM Implementation . . . . .               | 78        |
| 6.3               | Further Research . . . . .                    | 78        |
| <b>7</b>          | <b>Conclusion</b>                             | <b>80</b> |
| <b>Appendix</b>   |                                               |           |
| <b>A</b>          | <b>Appendix: Scaling approach</b>             | <b>81</b> |
| <b>B</b>          | <b>Appendix: Source code</b>                  | <b>82</b> |
| <b>References</b> |                                               | <b>83</b> |

# List of Figures

|      |                                                                                                     |    |
|------|-----------------------------------------------------------------------------------------------------|----|
| 2.1  | Cosmos architecture (based on (Cosmos Network, 2022b)) . . . . .                                    | 7  |
| 2.2  | CFMM AMM example of Uniswap (swap execution) . . . . .                                              | 12 |
| 2.3  | Bonding curve example (based on (Balasanov, 2020)) . . . . .                                        | 13 |
| 2.4  | Augmented Bonding Curve example (taken form (Zargham et al., 2020))                                 | 14 |
| 2.5  | Undergirding bonding curve - illustrative undergirding example . . . .                              | 16 |
| 2.6  | Undergirding Bonding Curve - Bézier curves . . . . .                                                | 21 |
| 2.7  | Cubic Bézier curve example . . . . .                                                                | 23 |
| 2.8  | $b_i$ range according to the curvature conditions . . . . .                                         | 27 |
| 2.9  | $a_i$ range according to the curvature conditions . . . . .                                         | 28 |
| 3.1  | Information System Research Framework Application (Hevner et al., 2004) . . . . .                   | 42 |
| 4.1  | Minimum and maximum initial pool balance example . . . . .                                          | 46 |
| 4.2  | Minimum and maximum initial pool balance - fixed parameters . . . .                                 | 47 |
| 4.3  | Curve shape examples for same project parameters, initial pool balance and slope in $P_2$ . . . . . | 48 |
| 4.4  | Basic principle of the scaling approach . . . . .                                                   | 50 |
| 4.5  | Two examples for different pool balances of the scaling approach . . . .                            | 50 |
| 4.6  | Example for undergirding with one self-consistency loop . . . . .                                   | 54 |
| 4.7  | Factor $f_{xy}$ depending on the slope in $P_2$ . . . . .                                           | 54 |
| 4.8  | $a_1$ placement in its bounds depending on the slope in $P_2$ . . . . .                             | 55 |
| 4.9  | Effect of increasing the slope in $P_2$ while $f_{xy}$ is held constant . . . . .                   | 56 |
| 4.10 | Shifting up $P_2$ while $f_{xy}$ is held constant . . . . .                                         | 56 |
| 4.11 | Concept of shifting up $P_2$ with a flat region . . . . .                                           | 57 |
| 4.12 | Example - shifting up $P_2$ with a flat region . . . . .                                            | 59 |
| 4.13 | Example - Extrapolating S2 . . . . .                                                                | 59 |
| 4.14 | Example - Open-ended quadratic segment S3 . . . . .                                                 | 60 |
| 4.15 | Example - $P_2$ shiftup zoomed on S2 changes . . . . .                                              | 61 |
| 4.16 | UML class diagram of the <i>ubc</i> module . . . . .                                                | 63 |

# List of Tables

|     |                                                                   |    |
|-----|-------------------------------------------------------------------|----|
| 3.1 | Requirements for the UBC curve shape . . . . .                    | 43 |
| 3.2 | Requirements for the UBC-MM implementation . . . . .              | 43 |
| 5.1 | Requirements for the UBC-MM implementation - functional . . . . . | 69 |
| 5.2 | Requirements for the UBC-MM implementation - system . . . . .     | 71 |

# Acronyms

|               |                                              |
|---------------|----------------------------------------------|
| <b>ABCI</b>   | Application Blockchain Interface             |
| <b>AMM</b>    | Automated Market Maker                       |
| <b>BFT</b>    | Byzantine Fault Tolerance                    |
| <b>BMWi</b>   | Bundesministerium für Wirtschaft und Energie |
| <b>CEX</b>    | Centralized Exchange                         |
| <b>CFMM</b>   | Constant Function Market Maker               |
| <b>CRUD</b>   | Create Read Update Delete                    |
| <b>DAO</b>    | Decentralized Autonomous Organization        |
| <b>DeFi</b>   | Decentralized Finance                        |
| <b>DEX</b>    | Decentralized Exchange                       |
| <b>DLT</b>    | Distributed Ledger Technology                |
| <b>DSR</b>    | Design Science Research                      |
| <b>EoT</b>    | Economy of Things                            |
| <b>IBC</b>    | Inter-Blockchain Communication               |
| <b>ICO</b>    | Initial Coin Offering                        |
| <b>IoT</b>    | Internet of Things                           |
| <b>LP</b>     | Liquidity Provider                           |
| <b>PBFT</b>   | Practical Byzantine Fault Tolerance          |
| <b>PoS</b>    | Proof of Stake                               |
| <b>PoW</b>    | Proof of Work                                |
| <b>UBC</b>    | Undergirding Bonding Curve                   |
| <b>UBC-MM</b> | Undergirding Bonding Curve Market Maker      |
| <b>UML</b>    | Unified Markup Language                      |

# Abstract

The emergence of Web3 as the third generation of the world wide web strives to use tokens as financial assets, that are build in everything within a digital decentralized ecosystem. They are classified as complex systems with a cryptoeconomic system in their heart, that enables users to exchange the tokens. In order to finance such systems and distribute its token, initial coin offerings (ICO) were used in the past, that later traded on centralized and decentralized exchanges. These ICOs were mainly unregulated, leading to fraud and an official warning by the U.S. SEC towards companies offering future ICOs. The tokens traded on the exchanges, especially in Proof of Stake (PoS) systems, are highly volatile due to low liquidity available on exchanges compared to the total supply. This leads to market manipulation, token prices decoupling of their true value and companies shying away from these tokens.

These problems are encountered by a new, so called Undergirding Bonding Curve Market Maker (UBC-MM), proposed by Poddey and Lamberty making use of the recently emerged bonding curve concept. Potentially being at the heart of the cryptoeconomic system around Catena-X, the UBC-MM allows to finance projects using a low-volatile token with guaranteed liquidity. In contrast to classical market maker approaches, the value extracted by the UBC can be held within the system. The UBC-MM redistributes this value to its users by bending up (undergirding) the curve in the lower region, increasing the minimal token price for late sellers bringing psychological confidence into the system. In addition, the UBC-MM has further built-in security mechanism such as higher *pump and dump* and frontrunning resistance.

The UBC-MM was initially developed and patented by Robert Bosch GmbH. Thus, this work builds on the internal notebook of Poddey and Lamberty, uses the UBC-MM as prescriptive knowledge, solves open problems, and instantiates an initial implementation as part of a layer 1 blockchain. The first part of this work focuses on giving a holistic understanding of UBC-MM concept in order to contribute it to common knowledge. The second part derives the IT artefacts of this work by applying the design science research process by Peffers et al. (2006). The improvements, new fitting algorithm and instantiation of the UBC-MM being the IT artefacts are iteratively derived and evaluated. The third part evaluates the successful final instantiation of the UBC-MM using previously derived requirements. Putting all together, this work developed a new fitting algorithm for two Bézier curves to a given set of parameters and implemented the UBC-MM using the Cosmos SDK with sufficient numerical accuracy, being ready for operation.

# 1 Introduction

The third generation web, Web3 is an idea initially formulated by Gavin Wood in 2014 (Edelman, 2021). It strives to reinvent the internet by building a decentralized online ecosystem, that would use tokens as financial assets, which are build into anything online (Kharif, 2021). The Web3 is build on top of open technologies, networked computation, ubiquitous connectivity and intelligent web approaches. Due to the rise of crypto technologies, which are one of the central technologies for Web3, the emergence of a new Web3 is further accelerated. This especially effects the industry since, the Internet of Things (IoT) gets possible converted into an Economy of Things (EoT), that forms a digital economy of everything. The EoT corresponds to a complex system operated in open contexts, that establishes economic mechanisms for entities within the network (Poddey & Scharmann, 2019).

Having a look at complex systems, the economic mechanisms as a part of the Web3 and EoT space, can be described as a cryptoeconomic system. According to Zargham et al. (2020), cryptoeconomic systems are *digital data-driven economies facilitated by distributed ledger technology (DLT), such as blockchain, and making use of cryptographic token action as information carriers within the system* (Voshmgir, 2019; Zargham et al., 2020). As a subclass of socioeconomic systems, they are (1) dynamic because of the flow of assets and information through the network, (2) adaptive since they can adjust their behaviour based on the environment and (3) multi-scale by their local protocols.

In order to finance such cryptoeconomic systems and distribute its tokens, Initial Coin Offerings (ICOs) were used in the past. The concept of an ICO is similar to an initial public offering (IPO), raising capital to finance the project, whereas ICOs used unregulated exchange platforms to sell tokens (Kaal & Dell’Erba, 2017). Afterwards, the tokens are traded on bid/ask market makers in order to give everybody access to the token. As of today, the majority of tokens is traded on bid/ask markets with 24h volumes exceeding \$200B correlating to the total market cap (Chainalysis, 2022). In September 2020, the on-chain trading volume of Decentralized Exchanges (DEXs) first surpassed the volume of Centralized Exchange (CEX), with a peak dominance of more than 80% in June 2021 (Chainalysis, 2022). These statistics demonstrate the increasing importance of DEXs and the underlying Automated Market Maker (AMM)

mechanisms.

Recently, bonding curves gained a lot of attention in the cryptoeconomic community, being a mechanism for funding and distributing tokens in cryptoeconomic systems (Rouviere, 2019). Possibly replacing ICOs, they are a powerful tool, acting as a market maker, thus providing funding and liquidity (Zargham et al., 2020). Bonding curves map the price of the token to the supply of the token. Thus, the price only increases when the supply increases as well. They offer a price discovery mechanism, that is completely transparent and hard to get manipulated (Coutts, 2019). As argued by Lau (2018), money flowing into bonding curve system increases the *value* dollar-for-dollar. In contrast, the market capitalization is often increased by magnitudes, leading to *value*, that is magically created (Lau, 2018).

## 1.1 Problems

In nowadays Proof of Stake (PoS) systems, due to their security concept overcoming the energy consumption of Proof of Work (PoW), the majority of PoS-based blockchains have more than 50% of their total supply staked. Cosmos, Solana, Tezos, Algorand, Internet Computer and BNB Chain, to name some of the largest blockchains by market capitalization, even surpass the 70% mark as of May 2022 (Staking Rewards, 2022).

In combination with the increasing trading volume on DEXs, that also requires liquidity provisioning by locking tokens into the protocol, one could draw the conclusion, that only a very small percentage of the protocol tokens in a PoS-based blockchain are used for price discovery on open markets. In case the price discovery is based on bid/ask market markers (as the predominantly applied nowadays), the token price can be manipulated easier by smaller volumes, because of the limited liquidity being ad-hoc available in the market. This could also lead to higher price-volatility in the market due to the small percentage of supply being available for trading on exchanges. In addition, the token could get vulnerable to price manipulation such as *pump and dump* schemes. *Pump and dump* or also called scalping schemes manipulate the market by pumping the price artificially and selling cheaply purchased tokens at a later point in time, that will dump the market again. This kind of market manipulation is especially present in cryptocurrency markets as argued by Li et al. (2021) and causes investors to loose money (Li et al., 2021).

As argued by Farmer (2000), even highly liquid stock markets have a resistance for market efficiency in terms of the spread of price from (typically unknown) true value, with noticeable amplitudes even over long time frames. The resistance to market efficiency is increased by trend following patterns, being a common strategy positively correlated to recent price movements, and profit maximizing patterns (Farmer, 2000). Therefore it could be argued, that token prices could decouple even more from their true value due to the increasing influence of trend followers on illiquid markets, since they leverage price movements even more, causing the decoupling. These risks also apply for ICOs, that often attract huge trading volumes within seconds after their launch. As an example, the Basic Attention Token (BAT) raised \$35 million within 30 seconds after its launch (Kaal & Dell’Erba, 2017). This leads to massive pumps in the token price and dumps vice versa when the initial investors start to sell their token holdings. Other investors, therefore suffer the risk of major losses due to the initial pump or false information. Also the U.S. SEC recognized these risk and therefore officially warned investors on August 28, 2017 (Zetsche et al., 2017).

The tokens are traded on bid/ask market makers, that are predominately operated on CEXs and DEX. These market makers extract value by raising trading fees. A study by Chainalysis has found out, that the accumulated trading fees earned on the top five DEXs surpassed \$100 million within one month (Chainalysis, 2022). These trading fees are extracted and redistributed to the liquidity providers of the protocol. Thus, it can be argued, that there is an outflow of value, caused by the trading fees of DEXs and CEXs in order to enable the trading of the token.

## 1.2 Relevance

This work is part of the cryptoeconomic system design of Catena-X as an industry consortia funded by the German Bundesministerium für Wirtschaft und Energie (BMWi). Catena-X is an lighthouse application for **Gaia-X** as European project focused on collaborative data spaces (Gaia-x, 2022; Ganser, 2021). It strives to establish a collaborative value creation by multiple use cases such as traceability, CO<sub>2</sub> tracking, and live quality loops. Data chains and shared services are one of the central features that contribute to its success. The Catena-X ecosystem will be based on a Distributed Ledger Technology (DLT) that builds a new pillar for the emergence of Web3 (Kobylińska et al., 2021). One possible option could be, that the economic heart of the poten-

tial cryptoeconomic system created by Catena-X will be a market maker issuing the Catena-X token. A low-volatile token is targeted in order to overcome the main hurdle for widespread adoption of industrial companies entering the web3 world. Often these companies shy away from volatile tokens to reduce foreign currency risks, that are especially present in cryptocurrency markets. Furthermore, these companies require the tokens to have some resistance to market manipulations, that are also a currently an open problem (Lamberty & Poddey, 2020). In addition, companies using and building the system, want to participate in its success. A fair overall game theoretic tokenisation required, whereby the form of the market maker actually plays an important role; a fact which is mostly ignored in nowadays cryptoeconomic systems.

### 1.3 Contributions

This work contributes to the information systems domain by proposing, improving and implementing a new IT artefact solving the volatility, price manipulation and value extraction problems as described above and strengthens the incentive alignment among the actual contributors and users of the system. It contributes to the common knowledge in the space of cryptoeconomic systems, especially creating a new generation of bonding curves. This new generation is formed by the Undergirding Bonding Curve Market Maker (UBC-MM) as the artefact of this work. The basic concept of the UBC-MM was created and patented by Robert Bosch GmbH, but will be published as a open source project to contribute to common knowledge in the future. The UBC-MM is used for funding token-based projects, providing continuous liquidity and market making to its users. Its market making mechanisms aim to reduce artificial volatility (eg. pump and dump), while still allowing for a value growth, even if the majority of tokens is staked or locked in liquidity pools. This can be achieved, since the volatility and price elasticity are defined by the curve shape of the undergirding bonding curve. Market manipulations are prevented by increasing the amount of money needed to manipulate prices. Therefore also reducing the risk for early investors.

Within the scope of this work, the UBC-MM as an existing artefact of the prescriptive knowledge is described in detail. The fitting algorithm used for initializing a valid curve shape based on Bézier curves is evaluated and incrementally improved by multiple design cycles following the information systems research framework as part of the Design Science Research (DSR) process of Hevner et al. 2004. The goal is to improve

the existing UBC-MM concept to have a fitting algorithm calculating a curve shape, that is injective for a given set of project parameters at given restrictions. The first research question that this thesis addresses can thus be formulated as follows:

***RQ1:*** *How can two cubic Bézier curves be injectively fitted to a fixed set of project parameters at given restrictions?*

Furthermore, this work implements the UBC-MM as part of a layer 1 blockchain, making use of the Cosmos SDK, in order to evaluate and proof the feasibility of the UBC-MM. Since Cosmos SDK is given as the go-to blockchain framework used for Catena-X, the second research question can thus be formulated as follows:

***RQ2:*** *Can the UBC-MM be implemented using Cosmos SDK as a layer 1 blockchain, with adequate numerical accuracy?*

## 2 Literature Review

The literature review chapter strives to give a basic understanding of the prescriptive knowledge uses within this work. First the Cosmos Ecosystem and especially the Cosmos SDK are explained for the later UBC-MM implementation. Second the concept of automated market makers (AMM), the actors and fundamental economics are described. The concept of bonding curves and two instantiations being the augmented bonding curve and the risk-adjusted bonding curve are outlined for completeness. The main part of the literature review is formed by the detail explanation and derivation of the UBC-MM approach as described by Poddey and Lamberty (2022). The UBC-MM is part of an internal notebook at Bosch and therefore taken as prescriptive knowledge for this work.

### 2.1 Cosmos Ecosystem

The Cosmos network interconnects many independent blockchains that implement apps and services, therefore it is also called *internet of blockchains*. In difference to Bitcoin and Ethereum, Cosmos utilized the PoS consensus mechanism based on Tendermint Core (Kwon & Buchmann, 2019). Therefore Cosmos uses the Practical Byzantine Fault Tolerance (PBFT) algorithm as its consensus mechanism, which is based on the Byzantine Fault Tolerance (BFT) problem introduced by Lamport et al. in 1980 (Buchman, 2016; Pease et al., 1980). The interconnection of all independent blockchains is powered by Tendermint zones, while the Cosmos Hub builds the first central point of connection. These hubs and zones use the **Inter-Blockchain Communication (IBC)** protocol to communicate with each other. This allows blockchains to bridge tokens to each other without the need of any third party bridge provider (Kwon & Buchmann, 2019). These independent application-specific blockchains bring multiple benefits for developers such as (1) flexibility in code execution and framework usage, (2) performance in means of separating applications on multiple interconnected chains, (3) security features such as audited virtual-machines, proven programming languages or individual cryptographic functions and (4) sovereignty over the entire chain by the stakeholders of the application (Cosmos Network, 2022a).

### 2.1.1 Cosmos Architecture

Cosmos utilized the core characteristics of a blockchain as it is a replicated deterministic state-machine. That state machine holds all states of the machine, while state transitions are triggered by transactions. The entire state machine is deterministic, ensuring that the machine terminates in the same final state, if the start state and the sequence of all individual transactions are identical. The state-machine represents the application layer of the blockchain, which is built by the developers using the Cosmos SDK (Cosmos Network, 2022b).

In order to replicate the state-machine to all blockchain nodes, the Tendermint Core takes care of the consensus and networking layer. The communication between the application layer and Tendermint Core is standardized via the so called Application Blockchain Interface (ABCI). The entire architecture is visualized in figure 2.1. Incoming transaction are received by the network layer and internally checked by the consensus layer. Valid transactions are passed to the application layer via ABCI. These transactions consist of one or more messages. The individual messages are processed by the application layer which returns, whether all messages contained in the transaction were successfully or not, to the consensus layer. Messages are application-specific and must therefore be implemented by the developer (Cosmos Network, 2022b).

The most important functionalities of ABCI are (1) *CheckTx* that checks if the received transaction is valid, (2) *DeliverTx* that passes the transaction to the application layer and (3) *BeginBlock* / *EndBlock* that allows the automatic execution of messages at the beginning and end of each block.

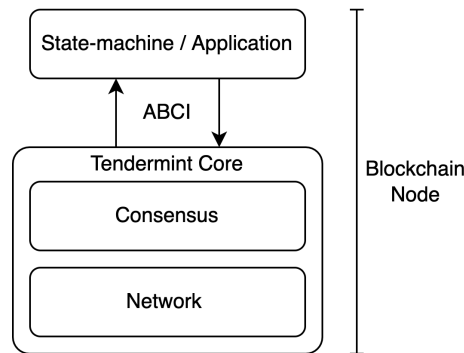


Figure 2.1: Cosmos architecture (based on (Cosmos Network, 2022b))

### 2.1.2 Cosmos SDK

The Cosmos SDK allows developers to create an application-specific blockchain by using the underlying components of the Cosmos network. Therefore the developers can focus on the application layer, while the consensus, network and inter-blockchain connection layers are handled by Tendermint Core (Cosmos Network, 2022c). Furthermore the Cosmos network offers a wide variety of modules such as *bank* for token transfer functionalities or *governance* for on-chain proposals and voting. Incoming messages from the Tendermint Core are routed to the respective modules, that implement the messages. These Cosmos modules as well as individually developed modules can be plugged into the customized blockchain (Cosmos Network, 2022b).

### 2.1.3 Ignite CLI

The Ignite CLI is a scaffolding tool, that is built on top of Cosmos SDK. It enables developers to build, test and launch application-specific Cosmos-based blockchains. Scaffolding creates the corpus for modules, messages, types and IBC packets which the developer can fill with functionality. Furthermore, the Ignite CLI allows developers to manage key, create validators, interact with the chains and the newly developed modules (Ignite Cli Docs, 2022b).

## 2.2 Automated Market Maker

Market makers are market participants, that concentrate liquidity and depth to markets in order to provide bid and ask offers to other market participants. Often market makers are large institutions such as brokerage houses that provide these services. Nevertheless individual traders can also be market makers if they are matching buy and sell offers with others. Market makers must provide continuous bid and ask quotes to market participants as well as the volume of the individual quote (Zhu et al., 2009). Typically market makers use order books in order to provide bid and ask quotes. These order books are an electronic list of all sell and buy orders that are open to be executed. Order books provide transparency for all market participants since they provide insight in prices, availability and depth of trade (Zhu et al., 2009).

In electronic markets, the Automated Market Makers (AMMs) gets increasingly important. As defined by Othman et al. (2013): *Automated market makers are algorithmic agents that enable participation and information elicitation in electronic markets*. Due to the absence of a centralized instance, AMMs became an integral part of Decentralized Finance (DeFi). They are the key element in nearly all Decentralized Exchanges (DEXs). In contrast to classical bid-ask order-book based exchanges they determine asset price algorithmically instead of matching buy and sell orders. The dominant category of AMMs used by DEXs are Constant Function Market Makers (CFMMs). In the following the term AMM will be used representative for AMM-based DEXs. CFMM as well as the actors and fundamental economics will be explained in the following (Xu et al., 2021).

### 2.2.1 Actors on AMM-based DEXs

In the case of an AMM, liquidity comes from Liquidity Providers (LPs). LPs can create liquidity pools and deposit their assets into these pools in return for pool shares. Since AMM-based DEXs are decentralized, anyone can create a new liquidity pool. The first LP creating the liquidity pool is called liquidity pool creator. When depositing liquidity into a pool, the LP gets pool shares in exchange. These pool shares allow the LP to exchange the shares back for the proportional share in the assets of the pool (Xu et al., 2021).

Liquidity pools consist of two or more assets, depending on their model. Initially AMMs were designed for exchanging risk assets, therefore liquidity pools used to have one risk asset and one base asset. The risk assets could be any token, while the base asset often is the protocol token, a stablecoin or the chains native coin. Token swaps for non-existing pairs are then routed through multiple liquidity pools in order to execute the swap.

Exchange users submit trades with their chosen exchange order to the AMM. By specifying the order of liquidity pools and the desired input and output asset, the AMM can route the trade through the corresponding liquidity pools. The exchange user is charged with a fee for each trade in the liquidity pools in order to compensate the LPs (Xu et al., 2021).

### 2.2.2 Fundamental Economics of AMMs

The economical goal of AMMs is to incentivize LPs to provide liquidity, while on the other hand side attracting exchange users to execute trades. In order to incentivize the LPs, AMMs offer (1) liquidity rewards as a share of the fees paid by exchange users, (2) staking rewards in form of AMM protocol tokens, (3) governance rights for LPs to participate in protocol design decisions (Xu et al., 2021).

The usage of AMM protocols brings three types of explicit costs.

1. **Liquidity withdrawal costs:** Some AMMs such as DODO use withdrawal fees to disincentivize LPs for withdrawing their liquidity. Less liquidity would lead the exchange users to higher slippage and therefore less attractiveness of the AMM.
2. **Swap fee:** Exchange users are charged with a swap fee for interacting with the liquidity pool. This percentage fee gets redistributed to the LP and the protocol itself. With this fee concept AMMs extract value as its done by other market makers such as CEXs (Xu et al., 2021). According to [intotheblock](https://app.intotheblock.com/)<sup>1</sup>, Uniswap extracts on average \$1.5M in swap fees per day, summing up to \$1.1B from their start in May 2020 up to May 2022.
3. **Gas Fee:** Since decentralised networks use some form of gas in order to pay for the execution of the transaction, exchange users and LPs are charged with a gas fee as well. Gas fees must be paid in the native tokens of the blockchains and can barely be influenced by the AMM.

Additionally to the explicit cost, there are implicit costs, that cannot be calculated in advance, due to complex system dynamics. These implicit costs are:

1. **Slippage:** The difference between the spot price and the realized price of a trade is called slippage. Slippage depends on the curve shape of the AMM and the volume of the transaction. Since the spot price never matches the realized price, slippage losses approach zero for infinitesimally small trades.
2. **Divergence loss:** LPs are exposed to volatility risks, since the AMM exchanged their provided assets. This causes the LP to hold numerically more of the lower performing asset, when withdrawing liquidity. Even if both assets are rising in price, the LP experiences losses compared to holding the assets outside of

---

<sup>1</sup><https://app.intotheblock.com/>

a liquidity pool. This phenomenon is called divergence loss or impermanent loss. Losses are only realized when withdrawing liquidity. Therefore, losses may disappear if asset prices move back to their original price ratio at the time liquidity is provided to the pool (Xu et al., 2021).

Furthermore, AMMs have additional costs that are caused by the oracle problem. The oracle problem describes the circumstance, that AMMs are prediction markets and therefore rely on external oracles which provide the actual exchange rate for the assets in the liquidity pools (Angeris & Chitra, 2020). Since using external smart contracts would bear risks of price manipulation, AMMs rely on arbitrageurs adjusting the exchange rates of the individual liquidity pools. Thus, arbitrageurs extract value of the liquidity pools, that form an additional cost for LPs (Evans, 2020).

### 2.2.3 Constant Function Market Makers

The actors and fundamental economics of AMMs can be implemented in so called Constant Function Market Maker (CFMM). CFMMs are classified as a subtype of AMMs, that determine feasible trades by their weighted reserves. Generally they can be represented by a state-space of a system  $\chi$  that consists of reserve quantities  $r_k$  of token  $k$ , the current spot exchange rate of each token  $p_k$ , the conservative functions invariant  $C$  and various hyper-parameters of the protocol  $\Omega$ . Putting all together, the system is described as:

$$\chi = (\{r'_k\}_{k=1,\dots,n}, \{p_k\}_{k=1,\dots,n}, C, \Omega) \quad (2.1)$$

State transitions can be expressed as:  $\chi \xrightarrow[f]{a} \chi'$ , whereas  $f$  is the state transition function and  $a \in A$  an action (Xu et al., 2021). Therefore exchanging assets through a liquidity pool (depositing one asset and withdrawing another one) is an action that can be expressed as:

$$(\{r_k\}_{k=1,\dots,n}, \{p_k\}_{k=1,\dots,n}, C, \Omega) \xrightarrow[f]{\text{swap}} (\{r'_k\}_{k=1,\dots,n}, \{p'_k\}_{k=1,\dots,n}, C, \Omega) \quad (2.2)$$

Note that the reserves and asset prices change, while the conservative functions invariant and other hyper-parameters are constant throughout the action (Xu et al., 2021).

The conservation function can be generally expressed as a function of the liquidity pool reserves and its conservation functions invariant. It can therefore be expressed as:

$$C_f = C(\{r_k\}) \quad (2.3)$$

For two asset pools, the conservation function is a concave, non-decreasing and non-negative graph approximation the x and y axes representing the respective asset reserves.

An example of a Uniswap liquidity pool executing a swap with 30 input token is shown in figure 2.2. The reserves of token1 are The conservation functions invariant is  $480,000$ . Since Uniswap uses an constant product approach, the conservation function is the product of the two reserves ( $r_1 \cdot r_2 = k$ ) (Zinsmeister & Robinson, 2020).

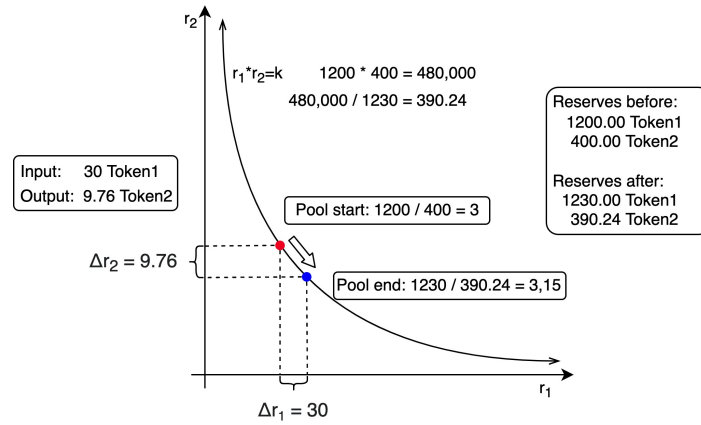


Figure 2.2: CFMM AMM example of Uniswap (swap execution)

## 2.3 Bonding Curves

As stated by Zargham et al. (2020) *Bonding curves are continuous liquidity mechanisms which are used in market design for cryptographically-supported token economies* (Zargham et al., 2020). The design space of bonding curves includes incentive mechanisms that enable new incentives for early adopters and a funding model for continuous organizations (Titcomb, 2020). Continuous organizations are a form of organisation similar to a Decentralized Autonomous Organization (DAO), that funnels the present and future cash-flows (Favre, 2018). In order to give tokens, that are bonded to the

bonding curve, an intrinsic value, the tokens represents a share of the future cash-flow of the continuous organization as it is with traditional stocks (Balasanov, 2020).

Bonding curves can be used as an AMM, whereas the exchange users are trading against the curve and not against other trades, as with established bid/ask market makers. As explained in the following, the bonding curve approach, different from CFMMs, guarantees liquidity per construction, therefore does not depend on liquidity providers.

The bonding curve corresponds to a graph with the token price on the ordinate (y-axis) and the supply on the abscissa (x-axis). The shape of bonding curves varies depending on the design decision of the project. Most often, projects choose to use an exponential form of bonding curve to increase the token price exponentially with an increasing token supply. The exponential form does not necessarily use Euler's number  $e$  as the base, often natural numbers are used in order to prevent numerical inaccuracies. The general bonding curve formula can be expressed as:  $y = m \cdot x^n$  with the token price  $y$ , token supply  $x$ , slope parameter  $m$  and exponent parameter  $n$ . The price related to buying or selling an amount of token is then determined by the area under the curve. Buying tokens against the bonding curve adds the value, provided by the buyer in exchange, into an escrow pool, that represents the pool reserves. Selling tokens vice versa, uses the escrow pool in order to payback value to the seller. Therefore the bonding curve offers guaranteed liquidity through the escrow pool, without having to pay rewards to liquidity providers. Figure 2.3 shows an example bonding curve and an exchange user buying some tokens (Balasanov, 2020).

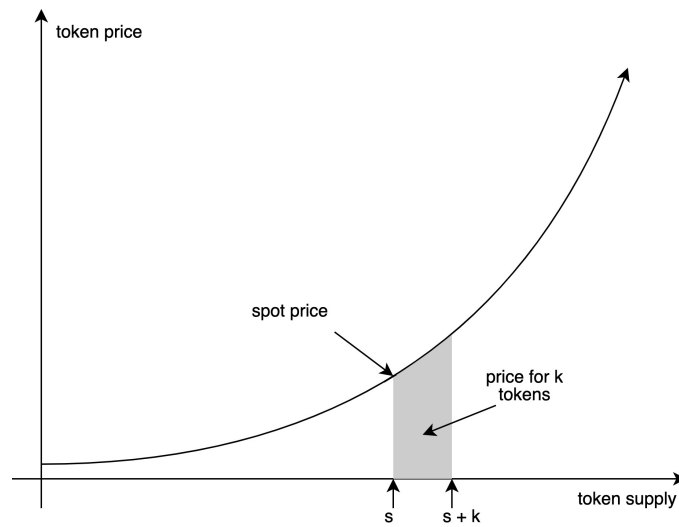


Figure 2.3: Bonding curve example (based on (Balasanov, 2020))

### 2.3.1 Augmented Bonding Curves

The augmented bonding curve is one well known instantiation of the bonding curve, that will be explained for completeness. The curve is static and will not be changed after the initialization. An example curve shape is visualized in figure 2.4. The token reserve  $R$  is represented by the area under the curve. The token supply  $S$  is the x-axis, while the community  $S$  is the subset of holders of  $S$ . The current spot price  $P$  is represented by the slope at the current supply. The bond-to-mint action is equivalent to buying tokens against the curve. The realized bond-to-mint price corresponds to the gradient triangle from the current supply to the posterior bond-to-mint supply. The realized burn-to-withdraw price corresponds the the gradient triangle form the current supply to the posterior burn-to-withdraw supply (Zargham et al., 2020).

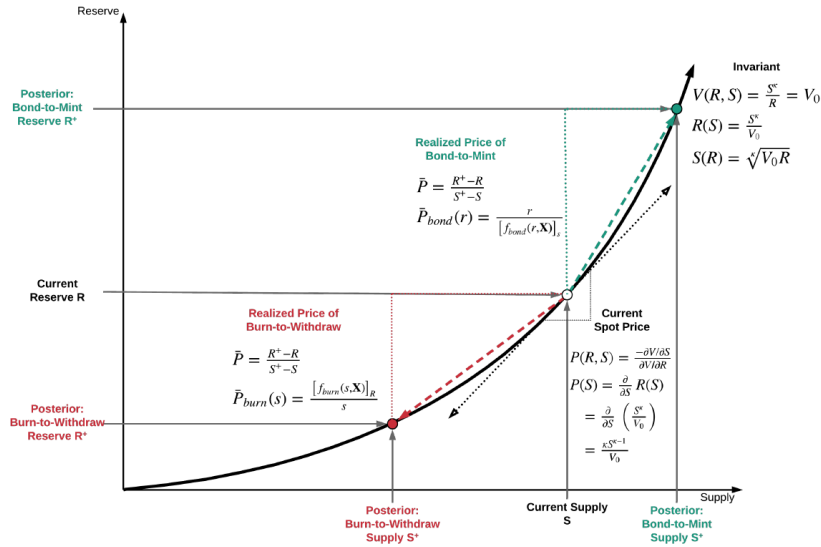


Figure 2.4: Augmented Bonding Curve example (taken from (Zargham et al., 2020))

### 2.3.2 Risk-adjusted Bonding Curves

Another bonding curve concept is the risk-adjusted bonding curve initially developed by BlockScience, ixo and the Interchain Foundation. The goal is to conceptualise a bonding curve, that dynamically adjusts to risk signals received by an internal prediction market. The prediction market is realized by users, that can attest their belief into the system. They can choose between positive and negative, while unattesting

or re-casting is not possible (BlockScience, 2022). In order to be dynamic, the risk-adjusted bonding curve adds a risk factor  $\alpha$ , that is considered as the probability of project success.  $\alpha$  is used as a co-efficient for the curve equation, therefore changes in  $\alpha$  steepen or flatten the risk-adjusted bonding curve. So it has a direct impact on the curve shape (BlockScience, 2022).

## 2.4 Undergirding Bonding Curve

*The Undergirding Bonding Curve Market Maker (UBC-MM) is a concept patented by Robert Bosch GmbH, that is currently within its grace period and therefore not published. The whole basic concept is part of an internal notebook by Alexander Poddey and Ricky Lamberty. This notebook is the primary reference for the following information regarding the UBC-MM in the literature review section. Other references apart of the notebook will be explicitly cited.*

The Undergirding Bonding Curve Market Maker (UBC-MM) uses the basic concept of bonding curves, while it enables the curve to change its shape at runtime. In the absence of undergirding, the pool balance stays constant as it is with standard bonding curves.

Undergirding means increasing the pool balance, therefore changing the shape of the curve, since the area under the curve increases, and vice versa. Changing the curve shape at runtime thereby needs to take care of frontrunning, which can be regarded as an attack extracting a part of the undergirded value. The current price at the trading point, as well as the characteristics (eg. slope) in the region around the current trading point, should not be changed to prevent frontrunning and users especially profiting from undergirding. Therefore, the curve shape is mostly changed in the lower region. In addition to frontrunning prevention, the undergirded value should reduce the risk to lose the invest in situations, where everybody dares to sell their tokens (all-exit risk). Thus the lower price for which they can sell is increased by an undergirding event.

The accumulated value on the one side would benefit the late sellers in an all exit scenario. On the other side, every user may become a late seller in a sudden exit avalanche. Therefore, the undergirding benefits every user. Due to the reduced risk to lose the invest in an all exit avalanche, early, anticipatory exit on price drops are

not necessary. The anticipatory exits in fact initiate and drive exit avalanches in established approaches. In contrast to that, the UBC-MM prevents the emergence of such avalanches in a self-reinforcing manner.

An example Undergirding Bonding Curve (UBC) is shown in figure 2.5. The left graph shows the initialized curve with its pool balance highlighted in grey under the bonding curve. The right graph shows the UBC after a (for illustrative purpose huge<sup>2</sup>) undergirding event, that increased the pool balance significantly. The current trading point, the slope of the curve in the current trading point and the part right to the current trading point stays unaffected.

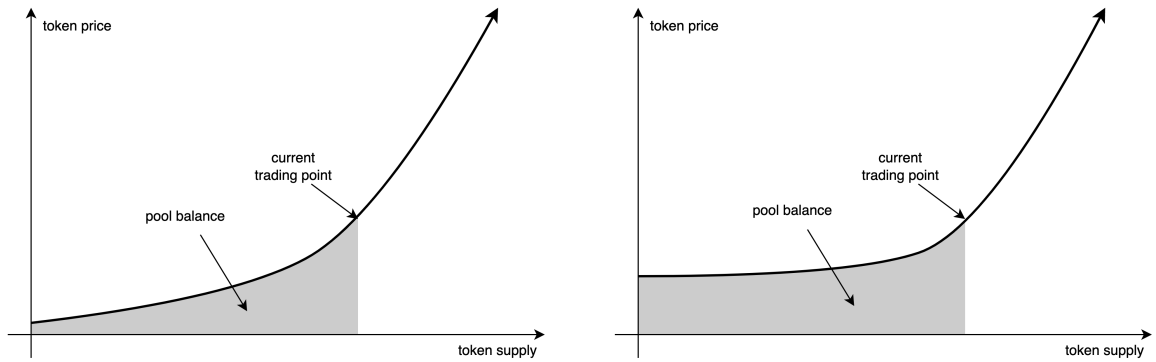


Figure 2.5: Undergirding bonding curve - illustrative undergirding example

To enable users to trade and provide a fully-backed reserve, the UBC-MM uses two tokens:

- The **project token**, that can be retrieved from the UBC-MM. The project token supply represents the x-axis of the UBC graph. The UBC-MM controls the supply of the project token, therefore minting and burning tokens depending on the interaction. The project tokens are fully-backed by euro voucher tokens and can always be redeemed by trading against the UBC.
- The **euro voucher token** represents a fiat pegged token, that is issued in a regulatory conform way<sup>3</sup>. The euro voucher token is used to buy project tokens and is send back to the seller, if project tokens are sold. Therefore, the pool balance of the UBC-MM is equivalent to the euro voucher holdings of the UBC-MM. This approach ensures, that the UBC-MM is always fully backed. The euro

<sup>2</sup>typically a large number of small-sized undergirding events would be applied over the lifetime of the project

<sup>3</sup>in a nutshell: it has not available for general use, but can only be send from the user to the bonding curve and back

voucher token could also be replaced by a third party stablecoin or other crypto assets, which would require a representation of the token on the underlying project chain having some regulative and technical complexities.

The project tokens should be fully functional according to the standard token of the used blockchain. For the later Cosmos implementation of the UBC-MM this results into market maker requirement **MM5**: *The UBC-MM integrates with the standard bank module of the Cosmos SDK.*

Since the UBC-MM is part of the protocol and forms the primary market for the tokens, extracted trading fees remain within the project and can be redistributed to the token holders. These trading fees are collected in a separate pool and can be reallocated to the pool balance for undergirding events. The primary market should effectively rail-guards the price range on secondary markets. Arbitrageurs lead the primary and secondary markets to close equilibrium within their no-arbitrage conditions, always balancing the prices.

The UBC-MM approach brings multiple benefits compared to standard token issuance that have their primary market on centralized bid/ask market makers. Especially while have the concept of PoS in mind, a large part of total supply is staked, reducing the available liquidity for bid/ask based market makers (Staking Rewards, 2022). The benefits of the UBC-MM are:

- **Low volatility** because the token price is related to the value attribution. In the case of exchange traded tokens, the price is vulnerable to decouple from the actual true value. Farmer showed in 2000, that even well-established and highly liquid markets such as stock markets often take years to decades for efficiency. This decoupling is caused by trend following and profit maximizing strategies (Farmer, 2000). PoS systems, that have the main proportion of tokens staked, are having a low ad-hoc liquidity on bid/ask markets. Therefore, the token price is much more influenced by short term trend following, leading to a higher volatility of the token. Small trading volumes could already change the token price, decoupling it from its true value, since value investors, that are assumed to stake their tokens do not actively participate in price discovery. In contrast, the UBC-MM allows users to trade against the bonding curve being always liquid. Due to the defined price elasticity (price action in relation to trading volume), sudden large price action leading to avalanches, disproportionate price increases or decreases are prevented. Furthermore, price movements are determined by

the slope at the current token supply. If the majority of investors belief in the system and stake their tokens, the token price cannot fall beyond the token price at the amount of staked tokens.

- ***Guaranteed liquidity*** since the UBC-MM increases the supply for buy actions and decreases the supply for sell actions vice versa. Because the price is related to the value attribution, even in avalanche situations, the price action is reasonable. Furthermore, the euro voucher tokens are held in escrow to ensure liquidity.
- ***Extracted value*** by the UBC-MM stays within the ecosystem and is redistributed to its token holders. CEXs and DEXs in comparison drain the extracted value for their own sake and only partially redistribute it to liquidity providers or shareholders. The value is therefore removed from the ecosystem.
- ***Design characteristics*** of the UBC-MM allow to prevent attacks such as *pump and dump* actions or front running. Furthermore the design characteristics add actions such as undergirding, that can be decided by a decentralized governance process.
- ***Interoperability*** with other exchanges and secondary markets. The UBC-MM builds the primary market, pinning the price elasticity for secondary markets.

Additionally, the UBC-MM reduces the risks for founders and early investors, that typically are at high risks to lose value related to token holdings. An exemplary reason would be that the system will not turn out to be useful. Because the UBC holds all investments within the pool balance, the investors could cash out at any time while having a guaranteed minimum return. This return might be smaller than the initial invest (when selling late), however the possible profit factor in case of the systems success is large. Therefore, the UBC-MM leads to an adequate risk/profit ratio.

Furthermore, when the system is used and becomes valuable, the undergirding increased the pool balance constantly proportional to its usage. The risk of founders and early investor therefore further decreases over time.

These two facts should stabilize the token price due to psychological reasons, since sell avalanches will be less likely because the token price has a lower bound that is steadily increased by undergirding events. In edge cases this could go as far as an early investor is having a guaranteed positive return rate for his initial investment, even if the system has a total failure after a few years of usage because of the accumulated value that is redistributed to the pool balance.

### 2.4.1 General Mechanisms of Undergirding Bonding Curves

The UBC must be able to change its curve shape during runtime in order to undergird the curve. To prevent frontrunning, directly profiting users, price and price elasticity changes, the UBC must preserve its curve shape in the region of the current trading point and beyond. Only the region left in adequate distance of the trading point is allowed to change. Otherwise, the previously mentioned problems could occur during the operation of the UBC-MM.

Furthermore, the UBC has a pre-described set of project parameters, that must be met. These project parameters include multiple reference points of the curve, the initial pool balance and two given slopes in order to define the price elasticity at the start of trading and at the point of the fully developed platform.

Using a stable second degree polynomial function would limit the UBC to only regard three informations when fitting the curve due to the number of free parameters in the generalized form of  $ax^2 + bx + c$ . Furthermore, changing the curve shape in the left region of the current trading point would also change the curve shape in the region of the trading point and beyond. Using polynomial function of a higher degree would add the risk of turning points within the curve, violating the strict up bend of the UBC and therefore effecting the trading of investors.

An exponential function of the form  $a \cdot b^{x+c} + d$  would give more freedom regarding the information taken into account for the curve fitting. Anyway, the freedom would not be sufficient for the amount of project parameters. Changing the curve shape in the lower region of the curve would also change the curve shape in the upper region, therefore effecting the current trading point. Furthermore, it is unclear, if using logarithmic functions would lead to numerical inaccuracies. Using multiple segments based on exponential functions was not analysed within the notebook, due to the complexity of solving logarithmic integral equations for one specific variable, the absence of logarithmic functions in blockchain math libraries and potential inaccuracies.

To fit a curve to multiple reference points and slopes, the UBC borrows the concept of Bézier curves from the domain of computer graphics. Bézier curves draw a curve through a given set of points on the curve, whereas the slope in these points can be controlled by the control points. In order to reduce the complexity, the UBC uses three segments, that are represented by cubic Bézier curves. The three segments as well as the points connection them are illustrated in figure 2.6 and described in the following:

- **Segment 0 ( $S0$ )** is used for the lower part of the UBC curve. The left sides boundary condition requires a zero slope condition, while the right side slightly increases the slope and therefore the token price.
- **Segment 1 ( $S1$ )** is used to connect Segment 0 with Segment 2. It bends up the curve to fit the flat slope of  $S0$  with the steep slope of  $S2$ .  $S0$  and  $S1$  are used for the undergirding events, increasing the area under  $S0$  and  $S1$  and thus flattening the bonding curve.
- **Segment 2 ( $S2$ )** is used for the upper part of the UBC. It has a steep slope, which increases the token price rapidly. The steep slope of  $S2$  is used to protect against 51% attacks. Since in a PoS system 51% of the tokens have to be held, an attack becomes more expensive as the supply of tokens increases. After  $S2$  would come an endless Segment 3.

The length and characteristic of these three Bézier segments are given by four points. These points have the following characteristics:

- **Point  $P_0$**  gives the price of the first token that is minted by the UBC. Undergirding events pull  $P_0$  up, decreasing the overall slope of  $S0$  and  $S1$ . The shift up of  $P_0$  should be the largest compared to all other points greater than  $P_0$ .
- **Point  $P_1$**  connects  $S0$  with  $S1$ . This point varies depending on the factor that determines the proportion of  $S1$  to  $S0$ .
- **Point  $P_2$**  marks the start of trading after the initial funding and connects  $S1$  with  $S2$ . The UBC assumes that at the start of the project 50% of tokens are in circulation. The x-value therefore marks 50% of the supply compared to the fully developed project.
- **Point  $P_3$**  marks the value of the fully developed project. The token price at  $P_3$  multiplied by the reference supply defines the value of the fully developed project. Assuming that 50% of the tokens are sold at the start of trading,  $P_3$  divided by  $P_2$  gives the profit factor for investors participating in the private sale.

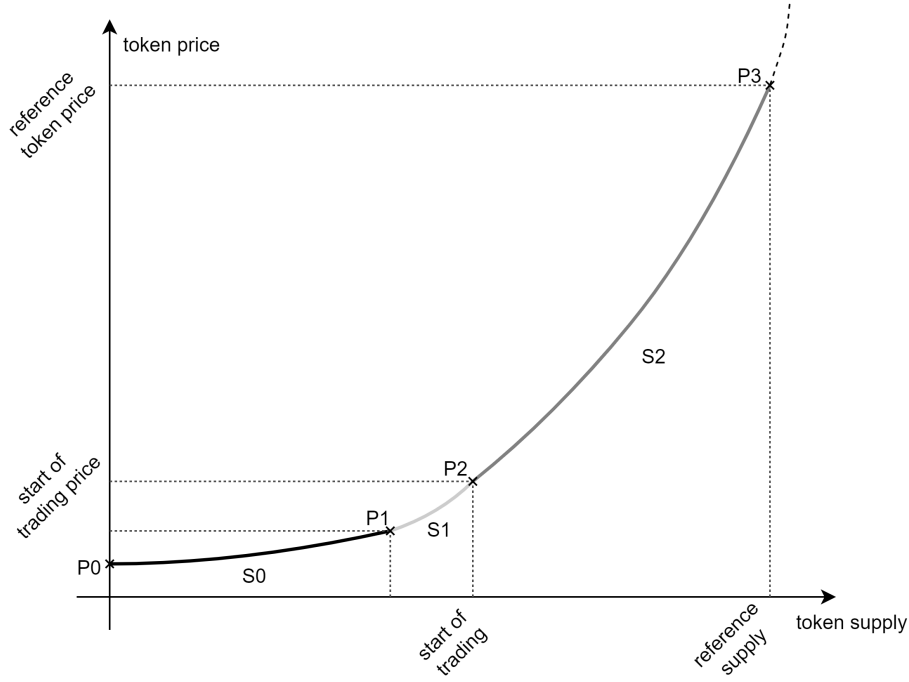


Figure 2.6: Undergirding Bonding Curve - Bézier curves

### 2.4.2 Bézier Curves

Bézier curves are a parametric representation of curves that is widely used in the computer graphics domain (Han et al., 2008). They are often implicitly known in computer graphics software as curves that can be modified by placing one or more control points. Bézier curves can be expressed in the Bernstein polynomial form as well as with the de Casteljau algorithm (Prautzsch et al., 2002). In this work, the Bernstein polynomial form is used for the sake of simplicity. The Bernstein polynomial form expresses a Bézier curve generally as:

$$P(t) = \sum_{i=0}^n B_i^n(t) \cdot P_i, \quad t \in [0, 1] \quad (2.4)$$

Where  $t$  is the parameter of the curve representation,  $n$  is the degree of the Bézier curve,  $P$  the control point and  $B(t)$  is the Bernstein polynomial that is expressed as:

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad \binom{n}{i} = \frac{n!}{i!(n-i)!} \quad (2.5)$$

A Bézier curve of the degree  $n$  holds  $n + 1$  control points, whereas the curve starts in  $P_0$  and ends in  $P_{n+1}$ . So these two points are always located on the curve, while all other control points influence the shape of the curve but are not necessarily located on the curve (Aflak, 2020).

Putting all together, a cubic Bézier curve with  $n = 3$ , and therefore 4 control points, can be express as (Aflak, 2020):

$$B(t) = (1 - t)^3 P_0 + 3t(1 - t)^2 P_1 + 3t^2(1 - t) P_2 + t^3 P_3 \quad (2.6)$$

This equation uses the actual points of the Bézier curve resulting in the individual points of the Bézier curve. It can also be split up into the x and y coordinates leading to the following equations:

$$x(t) = (1 - t)^3 x_0 + 3t(1 - t)^2 x_1 + 3t^2(1 - t) x_2 + t^3 x_3 \quad (2.7)$$

$$y(t) = (1 - t)^3 y_0 + 3t(1 - t)^2 y_1 + 3t^2(1 - t) y_2 + t^3 y_3 \quad (2.8)$$

This work uses a normalized form of Bézier curves, that can be derived as followed: General Bézier curves are based on individual points, the control points and therefore the curve can be placed freely in a coordinate system. Thus, these Bézier curves cannot be integrated, since the curve overlap itself or have multiple y-value for one x-value. As shown above, the x- and y-coordinate are parametrized functions of  $t$ . Therefore  $t$  cannot be easily substituted by  $x$ , since it would result in a complex term, without a close form solution.

A linear substitution is of  $t$  is only possible equidistant x-values of all inner and out control points. An example would be  $x_0 = 0, x_1 = \frac{1}{3}, x_2 = \frac{2}{3}, x_3 = 1$ . Thus, the  $x(t)$  graph forms a straight line allowing a linear substitution of  $t = ax$ ,  $a \in \mathbb{R}$ . This form of Bézier curves, that allow a linear substitution of  $t$  will be called normalized form. These normalized Bézier curves use the  $y(t)$  term of the slitted point equation and substitute  $t$  with  $x$  in order to calculate integrals and derivatives.

An example of a cubic Bézier curve is illustrated in figure 2.7. As it can be seen in this example, a straight line through  $P_0$  and  $P_1$  forms the tangent of the curve in  $P_0$  and thus the slope. This also applies to a straight line through  $P_3$  and  $P_2$ , which gives the tangent in  $P_3$ .

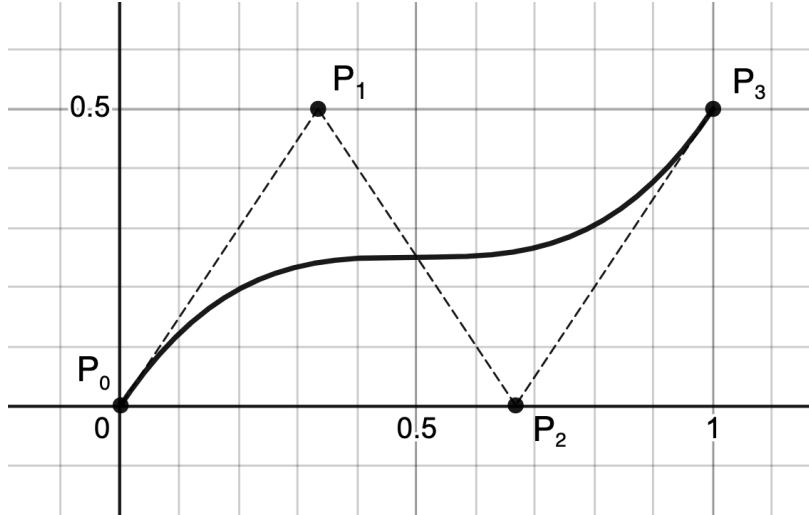


Figure 2.7: Cubic Bézier curve example

### 2.4.3 Basic Equations for Bézier Curves

The UBC fits cubic Bézier curves through predefined points  $P_i$ . Since the UBC maps a given token supply on the x-axis to a token price on the y-axis, the Bézier curve  $\Gamma$  must be defined as:  $\Gamma : x \rightarrow y$ . Bézier curves are a parametrized representation. Because the UBC uses the token supply  $x$  as input and not the parameter  $t$ , a substitution of  $t$  must be performed. As shown in section 2.4.2, there is no closed form solution for general Bézier curves. The UBC therefore uses the normalized form of Bézier curves allowing an linear mapping of  $x \rightarrow t$  because of the linear characteristics of  $x(t)$ . The substitution term can be expressed as:

$$x(t) = (x_{i+1} - x_i) \cdot t = \Delta x_i t \quad (2.9)$$

$$t(x) = \frac{x - x_i}{\Delta x_i} \quad (2.10)$$

The UBC segment  $i$  is expressed as  $\Gamma_i(t)$  representing the  $y(t)$  of the split Bézier curve form. The outer control points are  $P_i$  and  $P_{i+1}$  of the UBC. The two inner control points controlling the slope of the Bézier curve in the outer control points. These inner control points are named  $a_i$  for the left control point, controlling the slope in  $P_i$  and  $b_i$ , controlling the slope in  $P_{i+1}$ .

$$\Gamma(t) = (1 - t)^3 \cdot P_i + 3t(1 - t)^2 \cdot a_i + 3t^2(1 - t) \cdot b_i + t^3 \cdot P_{i+1} \quad (2.11)$$

Since the UBC must fit the slopes of the Bézier curves at the connection point  $P_i$ , the first derivation of the Bézier segment must be determined. Furthermore, the second derivative of the segment is needed to ensure continuous curvature and is used to enforce a curve without inflection point.

$$\Gamma'(t) = 3 \cdot [-(1-t)^2 \cdot P_i + (1-3t)(1-t) \cdot a_i + t(2-3t) \cdot b_i + t^2 \cdot P_{i+1}] \quad (2.12)$$

$$\Gamma''(t) = 6 \cdot [(1-t) \cdot P_i + (3t-2) \cdot a_i + (1-3t) \cdot b_i + t \cdot P_{i+1}] \quad (2.13)$$

In order to connect the segments smoothly, the slope in their connection point must match. Since the segment length in  $x$  can differ, the scaling of the slopes must be taken into account. The same applies for the curvature in the connection point. Thus it can be defined:

$$\Gamma'_i(t=1) \cdot \frac{\Delta x_i}{\Delta x_{i+1}} = \Gamma'_{i+1}(t=0) \quad (2.14)$$

$$\Gamma''_i(t=1) \cdot \frac{\Delta x_i}{\Delta x_{i+1}} = \Gamma''_{i+1}(t=0) \quad (2.15)$$

Since the UBC primarily depends on the slope at  $t=0$  and  $t=1$ , these values can be inserted into the first and second derivation to have them defined.

$$\Gamma'_i(t=0) = 3(-P_i + a_i) \quad (2.16)$$

$$\Gamma'_i(t=1) = 3(-b_i + P_{i+1}) \quad (2.17)$$

$$\Gamma''_i(t=0) = 6(P_i - 2a_i + b_i) \quad (2.18)$$

$$\Gamma''_i(t=1) = 6(a_i - 2b_i + P_{i+1}) \quad (2.19)$$

In order to determine the pool balance of the UBC, the area under the curve must be determined. Therefore the Bézier segment must be integrated in the real ( $x/y$ ) space, so  $t$  must be substituted in order to depend on the token supply  $x$ . Inserting equation 2.9,  $dx$  can be substituted to  $\Delta x_i \cdot dt$ . This results into:

$$\int \Gamma_i(x) dx = \Delta x_i \cdot \int \Gamma_i(t) dt \quad (2.20)$$

$$\begin{aligned}
\Delta x_i \cdot \int \Gamma_i(t) dt &= \Delta x_i \cdot \int (1-t)^3 \cdot P_i + 3t(1-t)^2 \cdot a_i + 3t^2(1-t) \cdot b_i + t^3 \cdot P_{i+1} dt \\
&= \Delta x_i \cdot \frac{1}{4} [(-t^4 + 4t^3 - 6t^2 + 4t) \cdot P_i + (3t^4 - 8t^3 + 6t^2) \\
&\quad \cdot a_i - (3t^4 - 4t^3) \cdot b_i + t^4 \cdot P_{i+1}] + C
\end{aligned} \tag{2.21}$$

#### 2.4.4 Curvature Conditions

The UBC must fit Bézier curves through the predefined points  $P_i$ , connecting them smoothly without any kink. To obtain an strictly bent up shape of the UBC over all three segments, the UBC must have a increasing slope and a positive curvature over the entire curve. If the curve starts with a slope larger or equal than zero and it is ensured, that there is always a positive curvature, the slope can be neglected in the conditions, since it is indirectly forced to be larger than zero. Therefore, the basic curvature condition for the UBC curve shape is given as:

$$\Gamma_i''(t) = 6 \cdot [(1-t) \cdot P_i + (3t-2) \cdot a_i + (1-3t) \cdot b_i + t \cdot P_{i+1}] > 0 \tag{2.22}$$

Since all points  $P$  are given, the curvature condition can be used to determine upper and lower bounds for the placement of the control point  $a_i$ , if  $b_i$  is given and vice versa of  $b_i$ , if  $a_i$  is given.

#### **Prescribed $\Gamma_i'(t=0)(a_i)$**

A prescribed slope at the left side  $\Gamma_i'(t=0)(a_i)$ , which can be determined by equation 2.17,  $a_i$  can be expressed as:

$$\begin{aligned}
\Gamma_i'(t=0) &= 3(-P_i + a_i) \\
a_i &= \frac{1}{3} \Gamma_i'(t=0) + P_i
\end{aligned} \tag{2.23}$$

Insert this equation solved for  $a_i$  into the basic curvature condition of equation 2.22, the upper and lower bounds for  $b_i$  can be defined as:

$$\begin{aligned} 6 \cdot [(1-t) \cdot P_i + (3t-2) \cdot a_i + (1-3t) \cdot b_i + t \cdot P_{i+1}] &> 0 \\ -[(1-t) \cdot P_i + (3t-2) \cdot a_i + t \cdot P_{i+1}] &< (1-3t) \cdot b_i \\ P_i - 2tP_i - t\Gamma'_i(t=0) + \frac{2}{3}\Gamma'_i(t=0) - t \cdot P_{i+1} &< (1-3t) \cdot b_i \end{aligned} \quad (2.24)$$

$$@t = 0$$

$$P_i + \frac{2}{3}\Gamma'_i(t=0) < b_i \quad (2.25)$$

$$@t = \frac{1}{3}$$

$$\frac{1}{3}P_i + \frac{1}{3}\Gamma'_i(t=0) - \frac{1}{3}P_{i+1} < 0 \quad (2.26)$$

$$@t = 1$$

$$\begin{aligned} -P_i - \frac{1}{3}\Gamma'_i(t=0) - P_{i+1} &< -2 \cdot b_i \\ \frac{1}{2}P_i + \frac{1}{6}\Gamma'_i(t=0) + \frac{1}{2}P_{i+1} &> b_i \end{aligned} \quad (2.27)$$

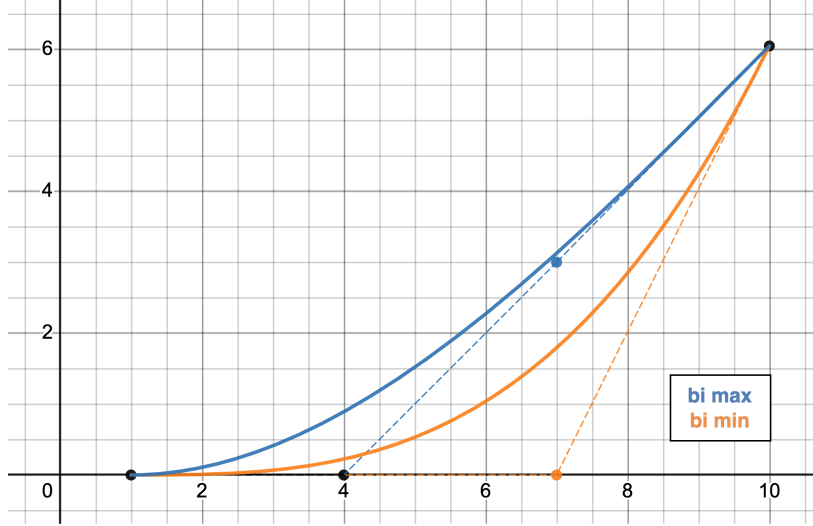
Summing up, condition 2.25 and 2.27 span a range in which  $b_i$  can be chosen. An illustrative example is shown in figure 2.8. The smallest possible value for  $b_i$  of equation 2.27 (orange line) would lead to the largest possible slope in  $P_{i+1}$  at  $t = 1$ . The largest possible value for  $b_i$  of equation 2.25 (blue line) would lead to the smallest possible slope in  $P_{i+1}$  vice versa, while ensuring no inflection (downbend region of the curve) occurs.

### **Prescribed $\Gamma'_i(t=1)(b_i)$**

A prescribed slope at the right side  $\Gamma'_i(t=1)(b_i)$ , which can be determined by equation 2.18,  $b_i$  can be expressed as:

$$\begin{aligned} \Gamma'_i(t=1) &= 3(-b_i + P_{i+1}) \\ b_i &= P_{i+1} - \frac{1}{3}\Gamma'_i(t=1) \end{aligned} \quad (2.28)$$

The same steps as for a prescribed  $a_i$  can be applied to get the lower and upper bound of  $b_i$ . So  $b_i$  is inserted into the basic curvature condition of equation 2.22 to get the

Figure 2.8:  $b_i$  range according to the curvature conditions

upper and lower bounds of  $a_i$ :

$$\begin{aligned}
 6 \cdot [(1-t) \cdot P_i + (3t-2) \cdot a_i + (1-3t) \cdot b_i + t \cdot P_{i+1}] &> 0 \\
 (t-1) \cdot P_i - (1-3t) \cdot b_i - t \cdot P_{i+1} &< (3t-2) \cdot a_i \\
 (t-1)P_i + \left(\frac{1}{3} - t\right)\Gamma'_i(t=1) + (2t-1)P_{i+1} &< (3t-2) \cdot a_i \quad (2.29)
 \end{aligned}$$

$$@t = 0$$

$$\begin{aligned}
 -P_i + \frac{1}{3}\Gamma'_i(t=0) - P_{i+1} &< -2 \cdot a_i \\
 \frac{1}{2}P_i - \frac{1}{6}\Gamma'_i(t=1) + \frac{1}{2}P_{i+1} &> a_i \quad (2.30)
 \end{aligned}$$

$$@t = \frac{2}{3}$$

$$\begin{aligned}
 -\frac{1}{3}P_i - \frac{1}{3}\Gamma'_i(t=1) + \frac{1}{3}P_{i+1} &< 0 \\
 P_i &> P_{i+1} - \Gamma'_i(t=1) \quad (2.31)
 \end{aligned}$$

$$@t = 1$$

$$-\frac{2}{3}\Gamma'_i(t=1) + P_{i+1} < a_i \quad (2.32)$$

These curvature conditions span a range for  $a_i$  as well. An example of the  $a_i$  range is illustrated in figure 2.9.

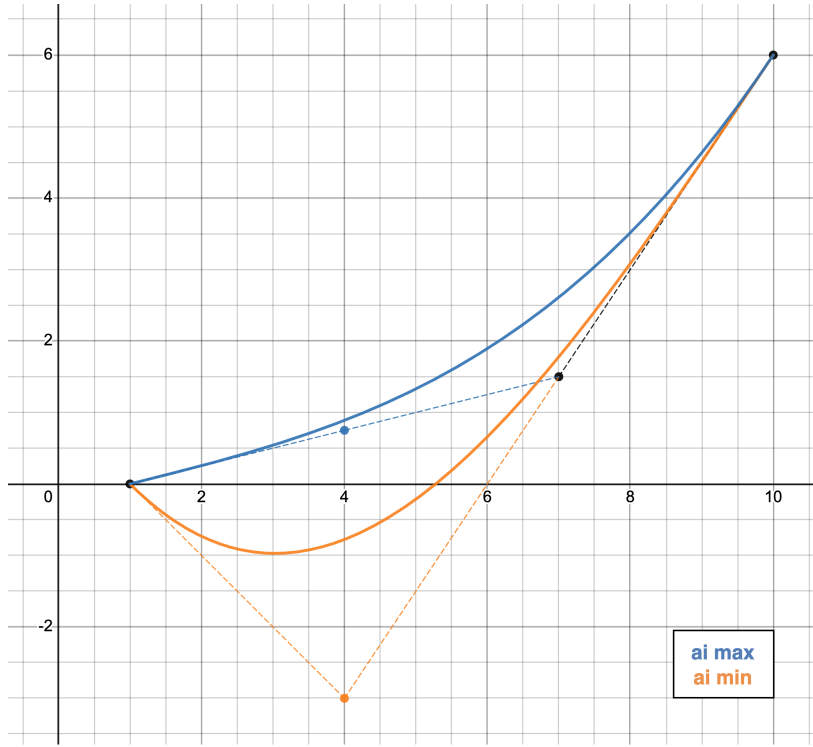


Figure 2.9:  $a_i$  range according to the curvature conditions

### 2.4.5 Fitting Two Segments

As discussed in section 2.4.1, the UBC must fit S0 and S1 to an initial pool balance, while S2 is given due to predefined parameters. Therefore  $P_3$  and  $P_2$  are given. Thus, the slope  $\Gamma'_2(t = 0)$  in  $P_2$  as well as the slope  $\Gamma'_2(t = 1)$  in  $P_3$  can be chosen within a pre-described range, but must result in curve shapes, that comply to the curvature conditions described section 2.4.4. The choices are often narrowed by the pool balance, in order to calculate a valid fit of S0 and S1.

#### Continuous Slope

To have a continuous slope across the entire UBC, the slope  $\Gamma'_i(t = 1)$  must match the slope  $\Gamma'_{i+1}(t = 0)$ . Since the UBC substitutes  $t$  to  $x$ , while the segment lengths differ, the slope must be scaled accordingly as showed in equation 2.14. Having the scaled slope as well as right connection point  $P_{i+1}$ ,  $b_i$  and  $P_{i+1}$  can be calculated by inserting

equation 2.23 and 2.28:

$$\begin{aligned}\Gamma'_i(t=1) \frac{\Delta x_{i+1}}{\Delta x_i} &= \Gamma'_{i+1}(t=0) \\ (-b_i + P_{i+1}) \frac{\Delta x_{i+1}}{\Delta x_i} &= (-P_{i+1} + a_{i+1})\end{aligned}\tag{2.33}$$

$$b_i = P_{i+1} \left(1 + \frac{\Delta x_i}{\Delta x_{i+1}}\right) - a_{i+1} \frac{\Delta x_i}{\Delta x_{i+1}}\tag{2.34}$$

$$P_{i+1} = \frac{a_{i+1} + \frac{\Delta x_{i+1}}{\Delta x_i} b_i}{\frac{\Delta x_{i+1}}{\Delta x_i} + 1}\tag{2.35}$$

### Pool Balance

The UBC-MM starts trading after the initialisation at  $P_2$ , therefore the pool balance is given by the area under S0 and S1, that must be provided before start of trading. Since the initial pool balance is predefined, the area under S0 and S1 must exactly fit the pool balance. The free parameters  $P_1$ ,  $P_0$ ,  $a_0$ ,  $b_0$ ,  $a_1$  and  $b_1$  must be chosen accordingly to exactly match this pool balance. The initial pool balance  $B_{pool}$  can be expressed as:

$$B_{pool} = B_{\Gamma_0} + B_{\Gamma_1}\tag{2.36}$$

The area under the curve is defined by its integral from 0 to 1. Thus, the integral equation 2.21 of the basics to Bézier curves is inserted as follows:

$$\begin{aligned}B_{pool} &= \Delta x_0 \cdot \int_0^1 \Gamma_0(t) dt + \Delta x_1 \cdot \int_0^1 \Gamma_1(t) dt \\ B_{pool} &= \frac{\Delta x_0}{4} [a_0 + b_0 + P_1] + \frac{\Delta x_0}{4} P_0 + \frac{\Delta x_1}{4} [a_1 + b_1 + P_2] + \frac{\Delta x_1}{4} P_1 \\ B_{pool} &= \frac{\Delta x_0}{4} [a_0 + b_0] + \frac{\Delta x_0}{4} P_0 + \frac{\Delta x_1}{4} [a_1 + b_1 + P_2] + \frac{\Delta x_0 + \Delta x_1}{4} P_1 \\ 4B_{pool} &= \Delta x_0 [a_0 + b_0] + \Delta x_0 P_0 + \Delta x_1 [a_1 + b_1 + P_2] + (\Delta x_0 + \Delta x_1) P_1\end{aligned}\tag{2.37}$$

### 2.4.6 Initializing the UBC

For initializing the UBC, S0 and S1 must be fitted accordingly to the predefined variables. To remove one free parameter and having a flat start of the UBC, it requires a zero slope at  $P_0$ .

$$\begin{aligned}\Gamma'_0(t=0) &= 0 \\ a_0 &= P_0\end{aligned}\tag{2.38}$$

Having this defined, a range of  $b_0$  depending on  $P_0$  and  $P_1$  can be expressed according to the curvature conditions (2.25, 2.27). Hence the range of  $b_{0,min}$  and  $b_{0,max}$  can be defined as:

$$\begin{aligned}P_0 + \frac{2}{3}\Gamma'_0(t=0) < b_0 < \frac{1}{2}P_0 + \frac{1}{6}\Gamma'_0(t=0) + \frac{1}{2}P_1 \\ P_0 < b_0 < \frac{1}{2}P_0 + \frac{1}{2}P_1\end{aligned}\tag{2.39}$$

In case of the initialization, the largest possible value for  $b_0$  will be chosen in order to have a more even slope and token distribution in  $P_1$ . After several undergirding events the UBC would approach  $b_{0,max}$  as well to fit more value under the curve. The effect of the parameter  $b$  on the slope was illustrated before in the above figure 2.8.

#### Factor $f_x$ and $f_y$

In order to control the ratio of the x and y length of S0 to S1, the UBC introduces two new parameters  $f_y$  and  $f_x$ . They define the relation between  $P_0$  to  $P_1$  and  $x_0$  to  $x_1$  vice versa. Factor  $f_x$  is defined as:

$$\begin{aligned}x_1 - x_0 &= f_x(x_2 - x_0) \\ x_1 &= f_x x_2 + (1 - f_x)x_0\end{aligned}\tag{2.40}$$

The same applies for factor  $f_y$  that is defined as:

$$\begin{aligned}P_1 - P_0 &= f_y(P_2 - P_0) \\ P_1 &= f_y P_2 + (1 - f_y)P_0\end{aligned}\tag{2.41}$$

Using factor  $f_y$ ,  $P_1$  can be eliminated in equation 2.39 for  $b_{0,max}$ :

$$\begin{aligned}
 b_{0,max} &= \frac{1}{2}P_0 + \frac{1}{2}P_1 \\
 b_{0,max} &= \frac{1}{2}P_0 + \frac{1}{2}f_yP_2 + \frac{1}{2}(1 - f_y)P_0 \\
 b_{0,max} &= (1 - \frac{1}{2}f_y)P_0 + \frac{1}{2}f_yP_2
 \end{aligned} \tag{2.42}$$

Having  $b_{0,max}$  defined and  $P_1$  eliminated,  $a_1$  can be expressed by using equation 2.35 and 2.41:

$$\begin{aligned}
 P_1 &= \frac{a_1 + \frac{\Delta x_1}{\Delta x_0}b_{0,max}}{\frac{\Delta x_1}{\Delta x_0} + 1} \\
 f_yP_2 + (1 - f_y)P_0 &= \frac{a_1 + \frac{\Delta x_1}{\Delta x_0}((1 - \frac{1}{2}f_y)P_0 + \frac{1}{2}f_yP_2)}{\frac{\Delta x_1}{\Delta x_0} + 1} \\
 (\frac{\Delta x_1}{\Delta x_0} + 1)f_yP_2 + (\frac{\Delta x_1}{\Delta x_0} + 1)(1 - f_y)P_0 &= a_1 + \frac{\Delta x_1}{\Delta x_0}((1 - \frac{1}{2}f_y)P_0 + \frac{1}{2}f_yP_2) \\
 a_1 &= (\frac{1}{2}\frac{\Delta x_1}{\Delta x_0} + 1)f_yP_2 + (\frac{1}{2}\frac{\Delta x_1}{\Delta x_0}f_y - 1 + f_y)P_0
 \end{aligned} \tag{2.43}$$

### Solving the Pool Balance for $P_0$

In order to fit S0 and S1, that the area under them is exactly the initial pool balance, the new variables must be inserted into the pool balance equation 2.37. Since the circular dependency of  $b_0$  is resolved into an implicit dependency by equation 2.43, all variables are now depending on given values. Therefore the pool balance equation can

be used to determine  $P_0$ . This results into:

$$\begin{aligned}
4B_{pool} &= \Delta x_0 b_0 + \Delta x_0 2P_0 + \Delta x_1 [a_1 + b_1 + P_2] + (\Delta x_0 + \Delta x_1) P_1 \\
4B_{pool} &= \left( \underbrace{4\Delta x_0 - \frac{3}{2}\Delta x_0 f_y - \frac{1}{2} \frac{\Delta x_1 \Delta x_1}{\Delta x_0} f_y}_{g_0} + 2\Delta x_1 - 2\Delta x_1 f_y \right) P_0 \\
&\quad - \left( -\frac{3}{2}\Delta x_0 f_y - \frac{1}{2} \frac{\Delta x_1 \Delta x_1}{\Delta x_0} f_y - 2\Delta x_1 f_y - \Delta x_1 \right) P_2 \\
&\quad + \Delta x_1 b_1 \\
4B_{pool} &= g_1 P_0 \\
&\quad - (g_0 - 2\Delta x_1 f_y - \Delta x_1) P_2 \\
&\quad + \Delta x_1 b_1
\end{aligned} \tag{2.44}$$

$$P_0 = \frac{1}{g_1} [4B_{pool} + (g_0 - 2\Delta x_1 f_y - \Delta x_1) P_2 - \Delta x_1 b_1] \tag{2.45}$$

### Algorithm for Fitting S0 and S1

Putting all together, an algorithm fitting S0 and S1 according to the given parameters can be defined. This algorithm can be uses to initialize the UBC.

#### Fitting S0 and S1

*INPUT:*  $B_{pool}, \underbrace{P_2, a_2, x_2}_{S2}, \underbrace{f_x, f_y}_{chosen\ freely}$   
*OUTPUT:*  $\underbrace{P_0, a_0, b_0, P_1}_{S0}, \underbrace{P_1, a_1, b_1}_{S1}$

*PROCESSING:*

---


$$\begin{aligned}
 x_1 &= f_x x_2 + (1 - f_x) x_0 \\
 g_0 &= -\frac{3}{2} \Delta x_0 f_y - \frac{1}{2} \frac{\Delta x_1 \Delta x_1}{\Delta x_0} f_y \\
 g_1 &= 4 \Delta x_0 + g_0 + 2 \Delta x_1 - 2 \Delta x_1 f_y \\
 b_1 &= P_2 \left(1 + \frac{\Delta x_1}{\Delta x_2}\right) - a_2 \frac{\Delta x_1}{\Delta x_2} \\
 P_0 &= \frac{1}{g_1} [4 B_{pool} + (g_0 - 2 \Delta x_1 f_y - \Delta x_1) P_2 - \Delta x_1 b_1] \\
 P_1 &= f_y P_2 + (1 - f_y) P_0 \\
 a_0 &= P_0 \\
 b_0 &= \frac{1}{2} P_0 + \frac{1}{2} P_1 \\
 &= \left(1 - \frac{1}{2} f_y\right) P_0 + \frac{1}{2} f_y P_2 \\
 a_1 &= \left(\frac{1}{2} \frac{\Delta x_1}{\Delta x_0} + 1\right) f_y P_2 - \left(\frac{1}{2} \frac{\Delta x_1}{\Delta x_0} f_y - 1 + f_y\right) P_0
 \end{aligned}$$


---

*MUST ENSURE:*

---


$$\begin{aligned}
 \frac{1}{2} P_1 - \frac{1}{2} \cdot \frac{\Delta x_1}{\Delta x_2} (-P_2 + a_2) + \frac{1}{2} P_2 &> a_1 \\
 -2 \cdot \frac{\Delta x_1}{\Delta x_2} (-P_2 + a_2) + P_2 &< a_1 \\
 P_1 &> P_2 - \frac{\Delta x_1}{\Delta x_2} 3(-P_2 + a_2) \\
 &\text{and} \\
 P_0 < b_0 &< \frac{1}{2} P_0 + \frac{1}{2} P_1 \\
 P_1 - P_0 &\geq \Gamma'_0(t=0) = 0
 \end{aligned}$$


---

Since the UBC-MM is predetermined to run on a blockchain, that comes with limited computing capacity, calculating a valid curve shape should not use a brute force approach. Therefore, this results into curve shape requirement **CS3**: *The curve shape can be fitted without using on-chain brute force parameter scans.*

Since the curvature conditions must be fulfilled, in order to have a strictly increasing curve shape with a positive curvature, this leads to curve shape requirement **CS2**: *The curve shape fulfils its curvature conditions everywhere for valid parameter sets.*

The actual UBC-MM implementation must ensure, that it only operates on valid curve shapes, that fulfil the curvature conditions. Since the fitting algorithm could produce invalid curve shapes, that must always been checked by the curvature conditions, this leads to the market maker requirement **MM7**: *The UBC-MM rejects invalid curve shapes and only operates on valid curve shapes.*

Additionally to the curvature conditions, the UBC-MM implementation must ensure, that the curve shape requirements are fulfilled in order to create valid curve shapes at all. This leads to market maker requirement **MM6**: *The UBC-MM fulfils the curve shape requirements, especially CS4 and CS5.*

## Parametrization of the UBC

The UBC uses three segments, whereas the first and second segment must be fitted to the initial pool balance and trading point. The curve shape is initially controlled by the parametrization of the third segment S2 and the initial pool balance. As discussed earlier in section 2.4.1, the UBC uses reference values of the project to determine curve parameters and hence initialize the curve. These reference values are:

- **Project value**  $\bar{v}$ : An approximation of the project value when it is fully developed.
- **Reference token price**  $\bar{p}$ : Token price when the project is fully developed.
- **Reference token supply**  $\bar{s}$ : Token supply when the project is fully developed
- **Profit factor**  $\bar{f}_{profit}$ : Return of invest for investors that bought the token at start of trading and selling at the reference price.
- **Initial pool balance**  $B_{pool}$ : Initial pool balance raised by founders, early investors and other private sales.

These project variables are interdependent and define the initial parametrization of the UBC. Assuming, that the start of trading is at 50 % of the token supply  $\bar{s}$ , these can be expressed as:

$$\bar{v} = \bar{p} \cdot \bar{s} \quad (2.46)$$

$$\bar{p} = \frac{\bar{v}}{\bar{s}} \quad (2.47)$$

$$\bar{s} = \frac{\bar{v}}{\bar{p}} \quad (2.48)$$

$$P_3(@\bar{s}) = \bar{p} \quad (2.49)$$

$$P_2(@\frac{1}{2}\bar{s}) = \frac{\bar{p}}{\bar{f}_{profit}} \quad (2.50)$$

Since the UBC-MM uses an open supply approach, the curve must continue after  $P_3$ . There are two options to continue: (1) using one or more Bézier segment or (2) using a linear or polynomial function that increases its slope continuously.

The final parameters for initializing the UBC are the slopes of the curve in  $P_2$  and  $P_3$ . Choosing these slopes directly impacts the shape of the UBC curve. These are also interdependent to the other reference values, since they are limiting each others choices.

- $\Gamma'_2(t = 1)$ : Defines the volatility and price action at the reference point. This value is crucial for the cost of an 51 % attack on the fully developed project. The steeper the slope, the more expensive the attack.
- $\Gamma'_2(t = 0)$ : Defines the volatility and price action at the initial trading point. The selected slope significantly influences the shape of the curve in the most used area of the curve. Its range is strongly limited by the chosen profit factor  $\bar{f}_{profit}$  and the initial pool balance. Since a low profit factor would lead to a higher value for  $P_2$ , a steeper slope would be required to fit small initial pool balances.

Overall, the project specific parameters must be chosen with care, due to their interdependence and the risks to render finding a curve impossible. The initialization of the UBC-MM should always result in the same curve shape for the same set of project parameters. This results into curve shape requirement **CS1**: *The curve shape is injective for the projects parameters  $\bar{s}$ ,  $\bar{p}$ ,  $\bar{f}_{profit}$ ,  $B_{pool}$ , slope in  $P_2$  and slope in  $P_3$  at given restrictions.* For each parameter set only one curve shape exists.

### 2.4.7 Undergirding Actions

As discussed in the basics on the UBC-MM in section 2.4, undergirding events are used to increase the pool balance represented by the area under S0 and S1. The goal is to preserve the curves shape around the current trading point and beyond. Therefore, undergirding events can only be executed, when the current trading point is beyond  $P_2$ . Changing the curve shape close to the trading point or beyond the trading point could result into users directly profiting from the undergirding event. As discussed in section 2.4, undergirding should reward late sellers, therefore the users should not directly profit from the undergirding event. A linear up shift of the curve would also result in direct profits for users, since the token price is increased for the entire curve. Furthermore, changing the curve shape would lead to uncertainties for investors, since slope changes would directly influence the price elasticity. This results into curve shape requirement **CS4**: *The curve can be undergirded by adding extracted value, preserving the characteristic of the curve in the region of the current trading point and beyond.* Undergirding actions re-fit S0 and S1 according to the algorithm defined in section 2.4.6, whereas the pool balance is increased by the pool balance  $B_{pool,Add}$  that should be undergirded. Thus, the new pool balance can be defined as:

$$B_{pool,New} = B_{pool} + B_{pool,Add} \quad (2.51)$$

This new pool balances  $B_{pool,New}$  is used as an input for the fitting algorithm. Following this procedure, undergirding would be limited to the extend, that S0 and S1 form a horizontal line with  $P_0 = P_1 = P_2$ . The UBC could be regarded as fully undergirded for the current position of  $P_2$

#### Shiftup $P_2$

To further undergird the curve,  $P_2$  and  $x_2$  must be shifted up, reducing the segment length of S2. Thus, the UBC can be further undergirded, until the maximum pool balance condition  $P_0 = P_1 = P_2$  is met again. This procedure is called  $P_2$  shiftup in the following. When shifting  $P_2$  by  $\Delta x_s$ , S2 must be recalculated by using the new slope  $\Gamma'_2(t = \frac{\Delta x_s}{\Delta x_2})$  for  $P_2$ . The re-fit of S0 and S1 also uses this new slope for the fitting algorithm as well as a further increased pool balance. The new pool balance must include the area under the curve within  $x_{2,old}$  and  $x_{2,new}$ . This additional pool balance is calculated equivalent to a buy action, which will be defined in the following

section. The new pool balance for undergirding the curve and shifting  $P_2$  is defined as:

$$B_{pool,New} = B_{pool} + B_{pool,\Delta x_s} \quad (2.52)$$

Since  $P_2$  shiftups will be executed, when the UBC-MM is already running and has undergirded value,  $P_2$  shiftups must preserve the characteristics of the curve. This leads to curve shape requirement **CS5**: *The curve can shift up  $P_2$ , preserving the characteristic of the curve in the region of the current trading point and beyond.*

### 2.4.8 Buy and Sell Actions

The UBC-MM enables users to buy and sell project tokens against the curve. Buying tokens corresponds to adding euro voucher tokens by increasing the area (pool balance) under the curve. Selling tokens corresponds to removing euro voucher tokens by decreasing the area and shifting the trading point. Both interactions move the trading point of the UBC-MM, which is always mapped to the current supply.

In order to prevent high frequency frontrunning of trading actions, the UBC-MM charges a trading fee, as its done by other market makers. The trading fee is a percentage share of the traded euro voucher tokens, that is subtracted of the input or output euro voucher tokens. Therefore, the UBC-MM accumulates euro voucher tokens, that can be used for later undergirding actions. This results into market maker requirement **MM3**: *The UBC-MM extracts a percentage trading fee.*

The trading fee is neglected in the following equations, since it is percentage share that gets subtracted of the input amount for buy transactions and output amount for sell transaction.

#### Buying against the curve

Following the leading approach of AMM-based DEXs, users are predominately using the buy exact amount feature: users provide an amount of euro voucher tokens they want to spend in exchange for as many tokens as possible. Applied on the UBC, the users provide the area they are willing to add to the pool balance  $v_{pb}$  in exchange for new tokens that are issued by the UBC  $\Delta s$ . The supply  $\bar{s}$  is therefore increased by

$\Delta s$  to the new supply  $\bar{s}_{new}$ . The buy action can be expressed as:

$$\begin{aligned}
 v_{pb} &= \int_{\bar{s}}^{\bar{s}+\Delta s} \Gamma_i(x) dx \\
 &= \Delta x_i \cdot \int_{\bar{t}}^{\bar{t}+\Delta t} \Gamma_i(t) dt \\
 &= \Delta x_i \cdot \frac{1}{4} \left[ \underbrace{(-t^4 + 4t^3 - 6t^2 + 4t) \cdot P_i + (3t^4 - 8t^3 + 6t^2) \cdot a_i - (3t^4 - 4t^3) \cdot b_i + t^4 \cdot P_{i+1}}_{F(t)} \right]_{\bar{t}}^{\bar{t}+\Delta t}
 \end{aligned} \tag{2.53}$$

Since the input value  $v_{pb}$  is given, this equation needs to be solved for  $\Delta t$ . Because the function is a 4th degree polynomial function, it cannot be solved easily. Thus, the function  $F(t)$  is approximated by a Taylor expansion. The Taylor series is a infinite sum function that approximates a function by its derivatives around the an expansion point. The new function is expressed as a Taylor polynomial of n-th degree. The largest possible degree depends on the differentiability of the function, while a higher degree leads to a more precise approximation. However, the Taylor series allows an approximation of the approximation error (Bronshtein et al., 2007). The Taylor expansion can be expressed as:

$$f(t, \bar{t}) \approx f(\bar{t}) + f'(\bar{t}) \cdot (t - \bar{t}) + \frac{1}{2} f''(\bar{t}) \cdot (t - \bar{t})^2 + \dots \tag{2.54}$$

Using the Taylor expansion, the integrand function  $F(t)$  can be approximated as:

$$\begin{aligned}
F(t) &= (-t^4 + 4t^3 - 6t^2 + 4t) \cdot P_i + (3t^4 - 8t^3 + 6t^2) \cdot a_i - (3t^4 - 4t^3) \cdot b_i + t^4 \cdot P_{i+1} \\
&\quad (2.55) \\
&\approx \underbrace{(-\bar{t}^4 + 4\bar{t}^3 - 6\bar{t}^2 + 4\bar{t}) \cdot P_i + (3\bar{t}^4 - 8\bar{t}^3 + 6\bar{t}^2) \cdot a_i - (3\bar{t}^4 - 4\bar{t}^3) \cdot b_i + \bar{t}^4 \cdot P_{i+1}}_a \\
&\quad + \underbrace{((-4\bar{t}^3 + 12\bar{t}^2 - 12\bar{t} + 4) \cdot P_i + (12\bar{t}^3 - 24\bar{t}^2 + 12\bar{t}) \cdot a_i - (12\bar{t}^3 - 12\bar{t}^2) \cdot b_i + 4\bar{t}^3 \cdot P_{i+1})}_{b} (t - \bar{t}) \\
&\quad + \underbrace{\frac{1}{2}((-12\bar{t}^2 + 24\bar{t} - 12) \cdot P_i + (36\bar{t}^2 - 48\bar{t} + 12) \cdot a_i - (36\bar{t}^2 - 24\bar{t}) \cdot b_i + 12\bar{t}^2 \cdot P_{i+1})}_{c} (t - \bar{t})^2 + \dots \\
&\approx a \\
&\quad + b(t - \bar{t}) \\
&\quad + c(t - \bar{t})^2 + \dots
\end{aligned} \tag{2.56}$$

Having a closer look at this expansion term, it can be noted, that the first part  $a$  nicely cancels with the lower limit of the integration. Furthermore, the insertion of  $t = \bar{t} + \Delta t$  nicely cancels  $\bar{t}$  for the entire term. These simplifications allow the term for determining  $\Delta t$  to be expressed as:

$$\begin{aligned}
4 \frac{v_{pb}}{\Delta x_i} &= b \cdot \Delta t + c \cdot \Delta t^2 \\
b \cdot \Delta t + c \cdot \Delta t^2 - 4 \frac{v_{pb}}{\Delta x_i} &= 0 \\
\Delta t_{1,2} &= -\frac{b}{2 \cdot c} \pm \sqrt{\frac{b^2}{4 \cdot c^2} + 4 \frac{v_{pb}}{c \cdot \Delta x_i}}
\end{aligned} \tag{2.57}$$

The term provides two approximated solutions for  $\Delta t$ . For buying tokens, only the larger one is of interest, because the smaller one would be subtracting pool balance instead of adding. Since the Taylor series allows to approximate the error, the initial estimation of  $\Delta t$  will be corrected by the approximated error. Furthermore, a  $f_{safety}$  (typically  $\approx 0.001$ ) is introduced, in order to ensure that the actually used  $\Delta t$  is a bit smaller than the numerically correct  $\Delta t$ . This mechanism prevents the UBC-MM of loosing value caused by small calculation errors. The used  $\Delta t_{used}$  is expressed as:

$$\Delta t_{used} = \left( 1 - \frac{F(t) - F(t, \bar{t})}{F(t)} - f_{safety} \right) \cdot \Delta t \tag{2.58}$$

The UBC-MM must enable its users to buy tokens, therefore this results into market maker requirement **MM1**: *The UBC-MM enables users to buy project tokens in exchange for euro voucher tokens.*

### Selling against the curve

A selling action takes an amount of tokens a users wants to sell  $\Delta s$  as input. The value the user gets back is equal to the area under the curve within the interval of  $\bar{s}$  and  $\bar{s} - \Delta s$ . The output amount can be expressed by the integral of equation 2.21 as:

$$\begin{aligned} v_{pb} &= \int_{\bar{s}-\Delta s}^{\bar{s}} \Gamma_i(x) dx = \Delta x_i \cdot \int_{\bar{t}-\Delta t}^{\bar{t}} \Gamma_i(t) dt \\ &= \Delta x_i \cdot \frac{1}{4} [(-t^4 + 4t^3 - 6t^2 + 4t) \cdot P_i + (3t^4 - 8t^3 + 6t^2) \cdot a_i - (3t^4 - 4t^3) \cdot b_i + t^4 \cdot P_{i+1}]_{\bar{t}-\Delta t}^{\bar{t}} \end{aligned} \quad (2.59)$$

Since the  $\Delta s$  is known, this equation can be easily solved for  $v_p b$ . This value is send to the user selling tokens, while the sold tokens are burned.

The UBC-MM must enable its uses to sell tokens against the curve, this results into the market maker requirement **MM2**: *The UBC-MM enables users to sell project tokens in exchange for euro voucher tokens.*

Overall the UBC-MM implementation must be able to calculate all equations above correctly. This must be ensured independently of the underlying data type, that is used for the actual implementation. This lead to the last market marker requirement **MM4**: *The UBC-MM is numerically accurate for all interactions.*

### 3 Research Method

This work uses a Design Science Research (DSR) approach (Hevner et al., 2004; Peffers et al., 2007) in order to derive the IT artefact and contribute to research and practice. Research rigor is achieved by following the DSR process defined by Peffers et al. (2007). Thus, this work derives its IT artefact by (1) identifying problems and outline its motivation, (2) defining objectives of this work, (3) design and development of the artefact, (4) demonstrating the results of the artefact in the current cycle, (5) evaluating these results and (6) communicating the final findings. The UBC-MM, being the most important existing artefact of the prescriptive knowledge, was analysed and traced in detail starting with the basics of bonding curves and Bézier curves in a mathematical point of view. Since the UBC-MM is patented by Robert Bosch GmbH and still in its grace period, it is neither published nor peer-reviewed. Thus, the overall UBC-mechanisms, its mathematical equations and design choices were validated during the literature review in order to identify possible pitfalls or design flaws at an early stage.

Following the DSR approach of Hevner et al. (2004), the basic structure of the information systems research framework and its application on this work is shown in figure 3.1. This work started with the relevance cycle outlining the relevancy of the UBC-MM in the context of Catena-X and its possible improvements on the organization. The requirements derived of the relevance cycle are summarized in table 3.1 and 3.2. The rigor cycle started by screening the knowledge base for relevant knowledge for the UBC-MM. Since the basic concept of the UBC-MM was already conceptualised beforehand by Poddey and Lamberty (2022), the UBC-MM in its initial status is seen as part of the knowledge base being prescriptive knowledge. Using analytical and experimental research methods, the UBC-MM concept and underlying hypothesis have been evaluated, and especially its mechanism for fitting the curve were improved within the design cycle.

The design cycle started by outlining the initial considerations about the UBC-MM such as limitations of the fitting algorithm and its minimum and maximum pool balance. Multiple approaches to fit the curve were designed, implemented and analysed in order to derive an efficient algorithm that fulfils the requirements. Created knowledge during the design cycles was transferred to the next cycle, in order to incrementally

improve the artefact. The final cycled focused on implementing the UBC-MM, using the previously derived approach for fitting the curve, on Cosmos SDK as part of a layer-1 blockchain. In order to efficiently describe the created artefacts, Unified Markup Language (UML) class diagrams were used in combination with descriptive elements to avoid proprietary code snippets.

The final artefact being the fully implemented UBC-MM with its newly derived fitting algorithm was evaluated in detail by executing invariance simulations and analytical case studies. These simulations and studies were focused on evaluating the numerical stability as well as proper functioning of the UBC-MM based on its requirements in table 3.2.

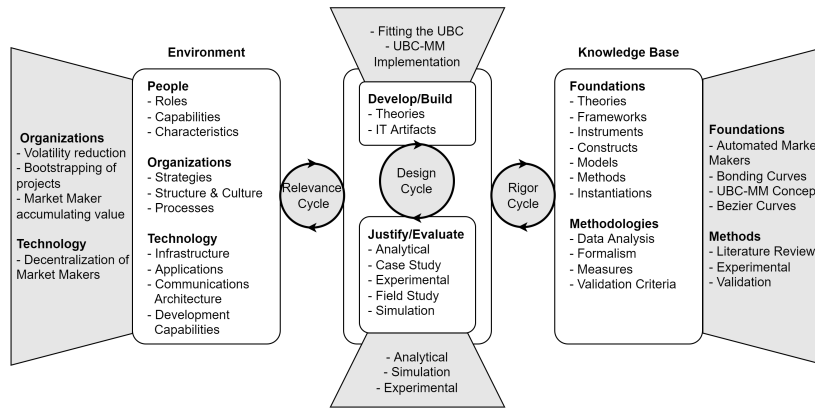


Figure 3.1: Information System Research Framework Application (Hevner et al., 2004)

## Paper structure

The structure follows the widely accepted DSR publication schema of Gregor and Hevner, 2013. Thus, (1) Introduction defines the scope and purpose as well as the relevance of this work, (2) Literature review section gives a brief overview to the existing descriptive knowledge and existing artefacts used within this work (3) Method section rationalises the research rigor, (4) Artefact description gives detailed insights into the artefacts mechanisms, design and research process as well as cycles, (5) Evaluation section ensures the rigorous design evaluation, (6) Discussion section outlines the learnings, novelty as well as closure of the research gap and (7) Conclusion section restates the findings, highlights and its relevance to research.

### Summary of requirements

The requirements for the curve shape and implementation of the UBC-MM were derived within the literature review section. The two tables 3.1 and 3.2 provide a summary of all requirements.

| No.        | Requirement                                                                                                                                                                                                                     |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CS1</b> | The curve shape is injective for the projects parameters $\bar{s}$ , $\bar{p}$ , $\bar{f}_{profit}$ , $B_{pool}$ , slope in $P_2$ and slope in $P_3$ at given restrictions. For each parameter set only one curve shape exists. |
| <b>CS2</b> | The curve shape fulfils its curvature conditions everywhere for valid parameter sets.                                                                                                                                           |
| <b>CS3</b> | The curve shape can be fitted without using on-chain brute force parameter scans.                                                                                                                                               |
| <b>CS4</b> | The curve can be undergirded by adding extracted value, preserving the characteristic of the curve in the region of the current trading point and beyond.                                                                       |
| <b>CS5</b> | The curve can shift up $P_2$ , preserving the characteristic of the curve in the region of the current trading point and beyond.                                                                                                |

Table 3.1: Requirements for the UBC curve shape

|            | No.        | Requirement                                                                             |
|------------|------------|-----------------------------------------------------------------------------------------|
| functional | <b>MM1</b> | The UBC-MM enables users to buy project tokens in exchange for euro voucher token.      |
|            | <b>MM2</b> | The UBC-MM enables users to sell project tokens in exchange for euro voucher token.     |
|            | <b>MM3</b> | The UBC-MM extracts a percentage trading fee.                                           |
| system     | <b>MM4</b> | The UBC-MM is numerically accurate for all interactions.                                |
|            | <b>MM5</b> | The UBC-MM integrates with the standard bank module of the Cosmos SDK.                  |
|            | <b>MM6</b> | The UBC-MM fulfils the curve shape requirements, especially <i>CS4</i> and <i>CS5</i> . |
|            | <b>MM7</b> | The UBC-MM rejects invalid curve shapes and only operates on valid curve shapes.        |

Table 3.2: Requirements for the UBC-MM implementation

## 4 Artefact Description

Following the research approach for this work as described in section 3, this section focuses on describing the created IT artefact and its design cycles. The iteration sequence is inspired by following the sequences in systems engineering as mentioned by Wieringa (Wieringa, 2014). Thus, the process starts with outlining the initial problem of the fitting algorithm. Within three further iterations, this work focuses on one problem, describes its possible solutions and cherry-picks the best solutions to derive the IT artefact. Therefore, each iteration ends with a decision that is carried on in the next cycle. Created knowledge is therefore transferred to the next cycles. The process starts with cycle zero that outlines the initial considerations of the basic UBC-MM approach being a given artefact for this work according to (Gregor & Hevner, 2013). Cycle one, two and three focus on developing a new efficient fitting algorithm, that full fills the existing requirements. Cycle four implements the entire UBC-MM concept using the fitting algorithm derived in the cycles before. Cycle one, two and three directly evaluate its design decisions by experiments in order to provide an insightful understanding of the underlying problem. Within these cycles the mathematically described solutions for the problems were implemented in Golang<sup>1</sup> to evaluate their functionality. Cycle four implements the entire UBC-MM using Cosmos SDK, its evaluation is more complex and will therefore be done in the separate evaluation chapter 5.

*The source code of the fitting algorithm development as well as the source code of the UBC-MM instantiation using the Cosmos SDK can be found on the disk in the appendix B.*

### 4.1 Cycle Zero - Initial Considerations

The initial cycle zero focuses on the initial fitting algorithm as described in section 2.4.6. It describes the basic limitations that come with the UBC-MM in its initial concept status.

---

<sup>1</sup>Golang is the programming language used within the Cosmos SDK.

### 4.1.1 Fitting the Curve

The initial fitting algorithm as described in section 2.4.6 uses  $f_x$  and  $f_y$  as input parameters, in order to place  $P_1$  or  $x_1$  in relation to  $P_2/x_2$  and  $P_0/x_0$ . Thus,  $f_x$  determines the length of S0 to S1 in x, while  $f_y$  determines the height of S0 to S1 in y.

Both parameters directly influence the curve shape as well as the minimum and maximum pool balance that can be fitted under the curve. Assuming that the curve is initialized with its minimum pool balance, the range to its maximum pool balance determines the value that can be undergirded later without the need of shifting  $P_2$ . This potential range should be maximized in order to increase the freedom for possible pool balances that can be fitted under a curve.

#### Minimum pool balance

Having the curvature conditions in mind, the shape of a UBC, that fits a minimum initial pool balance relative to the given project parameters and slope in  $P_2$ , the curve would approximate the following shape: S0 would approximate a horizontal line with  $P_0 = 0$  and  $P_1 = 0$  in order to not contain any pool balance. Therefore,  $f_y \rightarrow 0$  in order to fulfil the  $P_0 = P_1 = 0$  condition. Since S0 forms a horizontal line, the entire  $\Delta y$  must be crossed by S1. Thus, a value as large as possible for  $f_x$  must be chosen in order to keep S1 as short as possible on the x-axis, increasing the length of S0, that does not take any pool balance. S1 being the segment including pool balance would therefore strive to be as small as possible, while still crossing the entire  $\Delta y$ . The bottom curve illustrated in figure 4.1 shows an example minimum initial pool balance. Further increasing the length of S0, to reduce the pool balance even more, would violate the curvature conditions in S1. This sweet spot for the minimal x length of S1 will be called  $x_{1,opt}$ . Summing up, the minimum pool balance can be achieved in the edge case of  $f_y \rightarrow 0$  and  $f_x \rightarrow \frac{x_{1,opt}}{x_2}$ .

#### Maximum pool balance

The maximum initial pool balance relative to the given project parameters and slope in  $P_2$  would result in a curve approximation the following shape: S0 would approximate a horizontal line with  $P_0 \rightarrow P_2$  and  $P_1 \rightarrow P_2$  in order to fit the maximum pool balance under the curve. To fulfil the  $P_0 = P_1 = P_2$  condition, it must be ensured, that

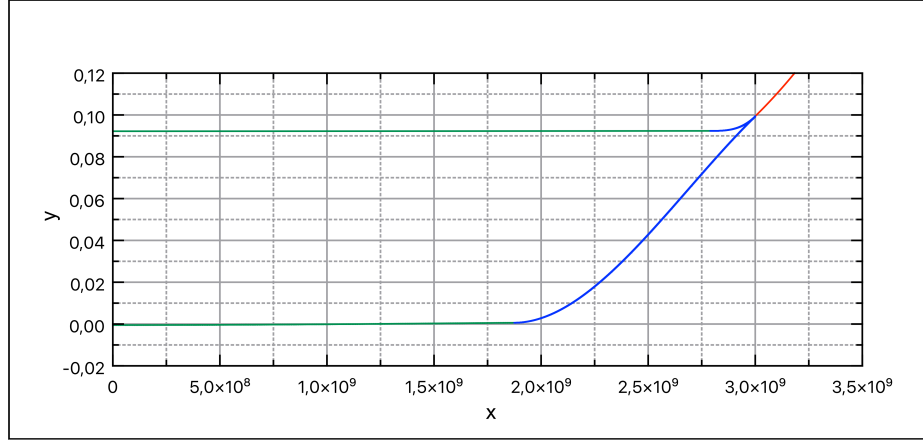


Figure 4.1: Minimum and maximum initial pool balance example

$f_y \rightarrow 0$ . Since  $\Delta y$  gets infinitesimally small,  $f_y$  can be neglected in the edge case. S1 would have to smoothly connect the horizontal line of S0 with S2 and its slope in  $P_2$ . Therefore S1 gets infinitesimally small in x and y just large enough to smoothly connect the zero slope of S0 with the slope of S2. Thus,  $f_x \rightarrow 1$  to maximize the length of S0. Summing up, the maximum pool balance can be achieved in the edge case of  $f_y \rightarrow 0$  and  $f_x \rightarrow 1$ .

## 4.2 Cycle One

Within cycle one, the initial fitting algorithm described in section 2.4.6 is implemented. This algorithm uses  $f_x$  and  $f_y$  as input parameters, in order to place  $P_1$  or  $x_1$  in relation to  $P_2/x_2$  and  $P_0/x_0$ . Thus,  $f_x$  determines the length of S0 to S1 in x, while  $f_y$  determines the height of S0 to S1 in y. Both parameters directly influence the curve shape and the minimum as well as maximum pool balance that can be fitted under the curve.

Since both parameters have a limited interdependent range that produces valid curve shapes, two options could be used to find a valid parameter set: (1) guessing based on experience and (2) full parameter sweep in order to brute force a valid set via *Trail and Error*.

Guessing based on experience allows to find valid parameter sets such as  $f_x = f_y = \frac{1}{5}$ , that produce valid curves in the majority of cases but also narrow down the range of minimum and maximum pool balances. This behaviour is shown figure 4.2, illustrating the maximum and minimum pool balance by keeping the parameter set. The black

curves illustrate the approximated theoretical boundaries as derived in section 4.1.1. *CS1* would be fulfilled, since there is only one possible fit for the curve shape. This fit could be achieved with one cycle, therefore not using an on-chain brute force approach and fulfilling *CS3*. As shown in figure 4.2, the pool balance range is limited, therefore there would exist a lot of curves, that would violate the curvature conditions if the pool balance exceeds the given range. This would violate *CS2*.

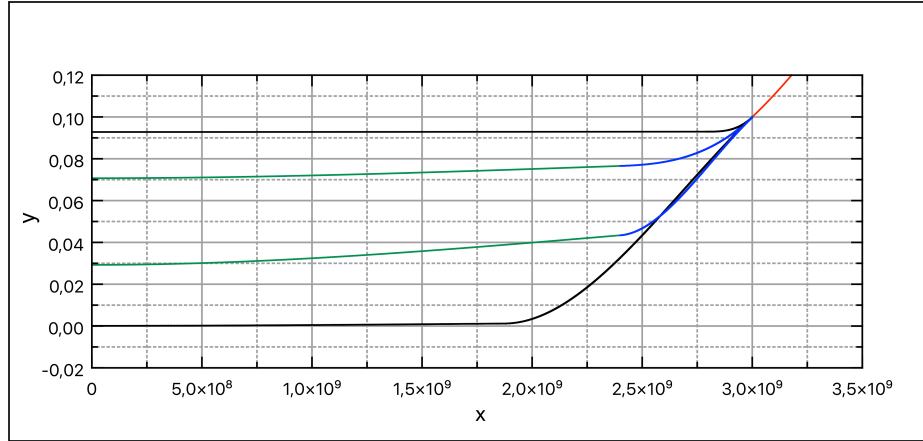


Figure 4.2: Minimum and maximum initial pool balance - fixed parameters

A full parameter sweep looping over all values for  $f_x$  and  $f_y$  would solve the problem of limited ranges of pool balances. This would require a nested loop, calculated on-chain, brute forcing one possible solution. Therefore, the full parameter sweep conflicts with requirement *CS3* by using a *Trail and Error* approach. Furthermore the full parameter sweep can return multiple results for a given set of project parameters. Thus, *CS1* is violated since the curve shape is not injective to its parameter set. The full parameter sweep results in multiple curve shapes for a fixed set of project parameters and a given slope in  $P_2$ , because  $P_1$  and  $x_1$  can be placed freely according to  $f_x$  and  $f_y$ . Figure 4.3 illustrates two curve shapes using two different sets of  $f_x$  and  $f_y$  both fitting for the same pool balance. *Curve shape 1* balances the pool balance more equal leading to a less steep (nearly horizontal) slope in  $S_0$ . *Curve shape 2* on the other hand side balances the pool balance closer to  $P_2$  leading to a more steep slope in  $P_2$ . Both curve shape are valid shapes with their own advantages. As an example: *Curve shape 1* rewards late seller, while *curve shape 2* rewards early sellers. Since discussing an optimal curve shape whether it should be as steep as possible or having a long horizontal area is out of scope of this work, a continuous curvature across the curve is regarded as optimal.

A basic definition of an optimal curve shape is done within the next research cycle together with a new approach on how to find a valid parameter set for the optimal curve shape.

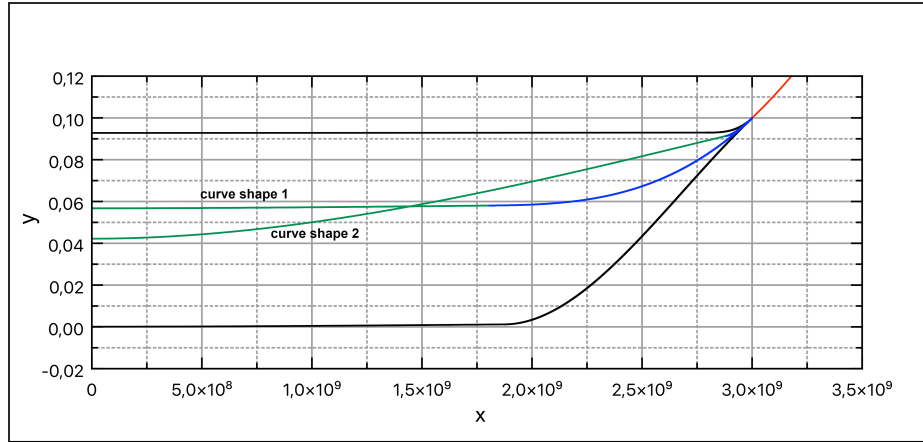


Figure 4.3: Curve shape examples for same project parameters, initial pool balance and slope in  $P_2$

### 4.3 Cycle Two

Cycle one has shown, that there can be multiple curve shapes with different characteristics fitting for a fixed set of project parameters. The various curve shapes have different pros and cons, whose detailed discussion is out of scope of this work. In addition, the curve shape freedom is undesirable from a project point of view. Therefore, an optimal curve shape condition has been derived in the following, resulting in a well determined shape. The optimal curve shape is based on the positioning of the control points  $a_i$  and  $b_i$ . To determine the optimal positioning, new slope conditions are introduced, to further narrow down the positioning. Having all control points optimally placed, the optimal curve, that only depends on  $P_2$ , can be calculated. However, this approach was discontinued because, after several iterations and considerable effort, it was found to be impractical. It remains unclear whether the error is a numerical one or an error in the derivation.

## Scaling approach

The scaling approach strives to provide an injective deterministic algorithm for fitting the curve using an optimal placement of  $b_1$ ,  $a_1$ ,  $b_0$  and  $a_0$ . Assuming that the optimal positioning of these control points can be calculated depending on each other, S0 and S1 can be scaled. Keeping the x to y ratio of the segments keeps their control points positioned optimally. This effect can be compared to scaling an image in computer graphics programs. As long as the ratio of x to y is kept, the image will be preserved and will not be distorted. The equations for the optimal positioning are provided in appendix A, while the following will describe the basic concept of the scaling approach. S1 has a fixed start and is therefore anchored in  $P_2$ . The x to y ratio can be expressed as a gradient triangle. This gradient triangle can be used to calculate a line on which  $P_1$  can be positioned without changing the optimal positioning of the  $b_1$  or  $a_1$ . This line can be seen in figure 4.4 as the red line. On the other hand side, S0 also has an optimal positioning. Here a gradient triangle for the x to y ratio can be defined as well.  $P_0$  is the anchor for S0. Since  $P_0$  is no fixed project parameter, it can be used to vary its y-position in order to move the line upwards or downwards. This is represented by the green line in figure 4.4. Shifting the line upwards or downwards can be seen in the example in figure 4.5. On this line  $P_1$  and  $x_1$  can be moved without changing the optimal positioning of  $a_0$  or  $b_0$ . The actual values of  $P_1$  and  $x_1$  are given by the intersection point of the S1 and S0 line. The intersection point represents the position of  $P_1$  in which S1 as well as S0 has an optimal positioning of their control points. The basic principle is illustrated in figure 4.4. This approach is also capable of fitting multiple pool balances by varying  $P_0$ , therefore moving the intersection point  $P_1$ . Two examples illustrating the fitting of multiple pool balances are displayed in figure 4.5.

This approach should be solvable since it only depends on one free variable  $P_0$ . During the scope of this work, the error in the equations created, could not be found. Since this approach can thus not be used for the UBC-MM implementation, the complete derivation and calculation of the scaling approach is provided in appendix A.

Theoretically, this scaling approach should fulfil *CS1*, since there would only be one intersection point. *CS2* would be fulfilled due to the wide range of possible pool balances, since the intersection point can move across the red line. Having solved the equations, the intersection point can be calculated, therefore the approach fulfils *CS3*.

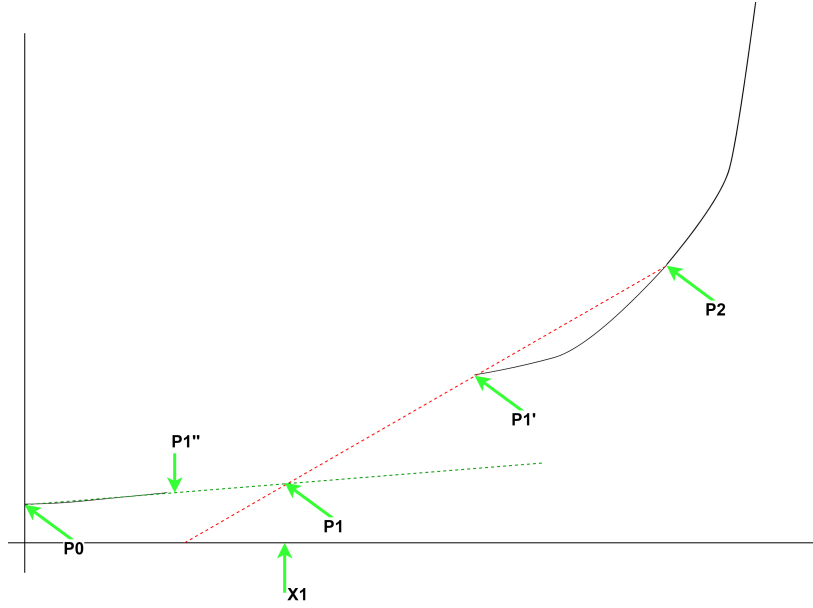


Figure 4.4: Basic principle of the scaling approach

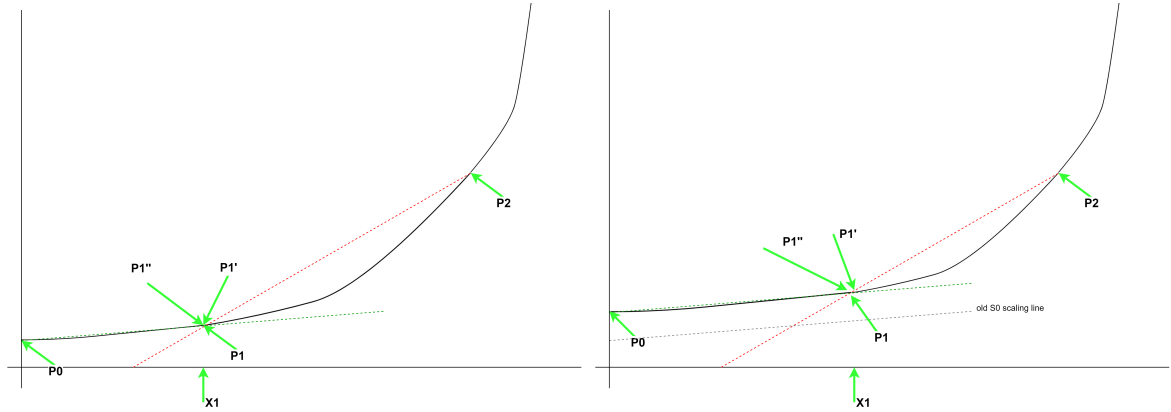


Figure 4.5: Two examples for different pool balances of the scaling approach

## 4.4 Cycle Three - Fitting algorithm

Since the scaling approach was not successful, this cycle continues to develop a feasible algorithm for fitting the curve. Approaching an optimal curve shape by placing the control points optimally is transferred to this cycle. However, this cycle only positions  $a_1$  optimally instead of all control points. This simplification can be used, because  $b_1$  is given by  $a_2$  point reflected in  $P_2$ . For  $b_0$  this approach sticks with the decision of the initial approach to use  $b_{0,max}$ , which can be well argued (see section 2.4.6). The gained knowledge of scaling segments in order to keep the optimal positioning of the

control points is transferred to this cycle. Therefore the x to y ratio of S1 is kept to position  $a_1$  optimally when adding pool balance. This x to y ratio will be called factor  $f_{xy}$ . It can be defined as:

$$f_{xy} = \frac{x_2 - x_1}{P_2 - P_1} \quad (4.60)$$

$$\begin{aligned} \Delta x_1 &= x_2 - x_1 \\ &\stackrel{!}{=} f_{xy}(P_2 - P_1) \end{aligned} \quad (4.61)$$

Inserting equation 2.41,  $P_1$  can be eliminated and  $x_1$  can be defined dependent of known variables:

$$\begin{aligned} \Delta x_1 &= f_{xy}(P_2 - f_y P_2 - (1 - f_y)P_0) \\ &= (f_{xy} - f_{xy}f_y)P_2 - (f_{xy} - f_{xy}f_y)P_0 \end{aligned} \quad (4.62)$$

$$x_1 = (f_x - f_x f_{xy})P_0 - (f_x - f_x f_{xy})P_2 + x_2 \quad (4.63)$$

#### 4.4.1 Initializing the Curve

In order to fit the curve, the equations of the initial fitting algorithm of section 2.4.6 must be adjusted, in order to optimally position  $a_1$ . First  $b_1$  can be defined by using the known variables of S2. Therefore  $b_1$  is defined by using equation 2.34 and inserting equation 4.63:

$$\begin{aligned} b_1 &= P_2(1 + \frac{\Delta x_1}{\Delta x_2}) - a_2 \frac{\Delta x_1}{\Delta x_2} \\ &= P_2(1 + \frac{(1 - f_{xy})P_2 - (1 - f_{xy})P_0}{\Delta x_2}) - a_2 \frac{(1 - f_{xy})P_2 - (1 - f_{xy})P_0}{\Delta x_2} \end{aligned} \quad (4.64)$$

Additionally  $a_1$  is defined by using equation 2.43 and inserting equation 4.63 and 4.62 vice versa:

$$\begin{aligned} a_1 &= (\frac{1}{2} \frac{\Delta x_1}{\Delta x_0} + 1)f_{xy}P_2 - (\frac{1}{2} \frac{\Delta x_1}{\Delta x_0} f_{xy} - 1 + f_{xy})P_0 \\ &= (\frac{1}{2} \frac{(f_{xy} - f_{xy}f_y)P_2 - (f_{xy} - f_{xy}f_y)P_0}{(f_{xy} - f_{xy}f_y)P_0 - (f_{xy} - f_{xy}f_y)P_2 + x_2} + 1)f_y P_2 \\ &\quad - (\frac{1}{2} \frac{(f_{xy} - f_{xy}f_y)P_2 - (f_{xy} - f_{xy}f_y)P_0}{(f_{xy} - f_{xy}f_y)P_0 - (f_{xy} - f_{xy}f_y)P_2 + x_2} f - 1 + f)P_0 \end{aligned} \quad (4.65)$$

Since the initial fitting algorithm of section 2.4.6 uses  $f_x$  and  $f_y$  as input. This approach sticks with  $f_y$  and uses the new  $f_{xy}$  in order to have an optimal positioning of  $a_1$ . Therefore,  $f_{xy}$  is another input for the algorithm, that directly influences the positioning of  $x_1$  as well of  $P_1$ . Using this relation,  $P_1$  can be determined if  $x_1$  is known and  $x_1$  if  $P_1$  is known vice versa. In contrast to the initial fitting algorithm,  $P_1$  must be known in order to calculate  $x_1$ . Since  $P_1$  depends on  $P_0$ , which itself is the actual result of solving the pool balance equation 2.45 requiring  $x_1$  as an input (in  $\Delta x_1$ ), this leads to a self-dependence. The equations of the initial fitting algorithm causing the self-dependence are:

$$\begin{aligned} x_1 &= f_{xy}(P_2 - f_y P_2 - (1 - f_y)P_0) \\ P_1 &= f_y P_2 + (1 - f_y)P_0 \\ P_0 &= \frac{1}{g_1} [4B_{pool} + (g_0 - 2\Delta x_1 f_y - \Delta x_1) P_2 - \Delta x_1 b_1] \end{aligned}$$

In order to solve this self-dependence, this approach relies on a precise first guess of  $x_1$ , that is initially calculated by a full parameter sweep using the positioning of  $a_1$  as a measurement of quality. The parameter sweep is used to determine the initial fit of the UBC and only calculated once off-chain before the initialisation. Subsequent adjustments can use this  $x_1$  and achieve the optimal positioning through self-consistency. This approach is described in more detail in the following context of undergirding. This approach fulfils CS1, since there is only one curve, that positions  $a_1$  optimally. Further substantiation for this is shown in figure 4.8. CS2 is fulfilled since the initial full parameter scan finds the corresponding  $f_{xy}$ , that places  $a_1$  optimally and fits the pool balance. A full parameter scan is only needed for initializing the curve, therefore the full parameter scan can be calculated once off-chain. Subsequent on-chain refitting can apply the self-consistency approach. Therefore this approach fulfils CS3.

#### 4.4.2 Undergirding

When executing undergirding actions,  $f_{xy}$  and  $f_y$  are held constant, since they represent a optimal curve shape, that can be scaled by  $f_{xy}$ . To increase the pool balance while keeping  $f_{xy}$ , the segment length of S1 is decreased, enlarging S0 vice versa. Therefore the curve shape is transforming from its initial curve shape to edge case as shown previously in figure 4.1, displaying the maximum pool balance. Keeping  $f_y$

for undergirding actions, while constantly decreasing the segment length of S1, this also shrinks  $\Delta y_0$ . Therefore transforming S0 from a steep segment as in the initial curve shape to a horizontal line in the edge case of S1 approaching zero. Using this approach, pool balance converges to the maximum pool balance in edge case without having to execute a full parameter sweep violating CS3.

Regarding the self-dependence of  $x_1$  in the fitting algorithm, that is solved by an initial full parameter sweep, this approach uses the old  $x_1$  as a precise first guess. Therefore, the self-dependence can be resolved by having this old  $x_1$  defined. Thus, the positioning of  $x_1$  is lagging behind by one fit and  $a_1$  could have a slight displacement of its optimal position, that depends on the size of the added pool balance. In order to eliminate this displacement, a self-consistency approach is used. Having calculated a new fit with  $x_1$  lagging behind, a better  $x_1$  can be calculated by using the position of  $P_1$  and  $f_{xy}$  as shown in equation 4.62. Using this better  $x_1$ , the UBC can be refitted without changing the pool balance to adjust all parameters, decreasing the displacement. Undergirding events only use the self-consistency approach and do not have to execute a full parameter scan.

Experiments using this approach have shown that approximately two self-consistency loops of refitting the UBC with the same pool balance are sufficient for having all parameters placed optimally. The displacement is justifiable smaller than 1% for large sized undergirding events. Extreme undergirding events would require three to four cycles to reach a smaller displacement. However, it can be argued that in normal operation of the UBC-MM a large number of smaller undergirding events would occur, that would reach its optimal placement within two self-consistency loops.

An example graph executing one self-consistency cycle is illustrated in figure 4.6. The first fit uses the old  $x_1$ , having a large displacement. The second fit calculates a better  $x_1$  with a major adjustment, that also further increases  $P_0$ . A third fit would only have minimal changes completely overlying the curve of the second fit and is therefore not displayed. Furthermore, partial mixing of the better  $x_1$  was tested to possibly increase the stability of the result. However, the mixing approach was discarded as using the self-consistency without mixing did not lead to any inconsistencies. Thus, mixing requires more cycles to converge without improving the result.

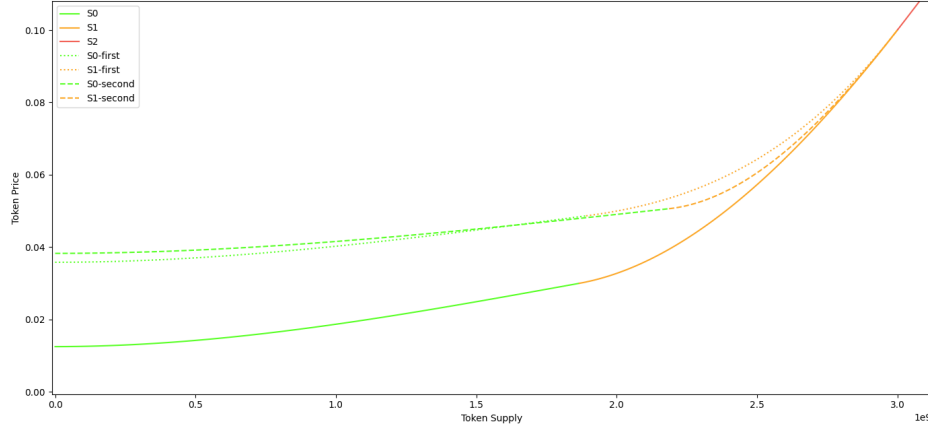
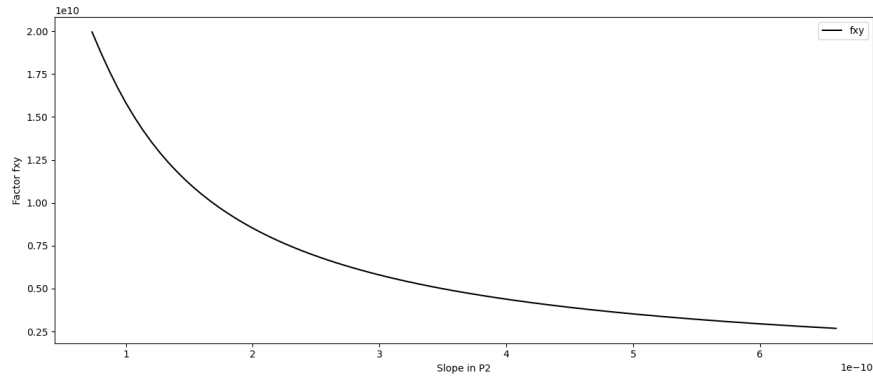


Figure 4.6: Example for undergirding with one self-consistency loop

### 4.4.3 Shifting up $P_2$

When shifting  $P_2$  on the curve, in contrast to standard undergirding discussed in section 2.4.7, the optimal value for the factor  $f_{xy}$  is not constant, but requires adoption. The reason lies in the displacement of  $a_1$  within its optimal position. The displacement is caused by the geometric change of the overall  $\Delta X$  to  $\Delta Y$  ratio change. Since the slope in  $P_2$  determines the ratio change,  $f_{xy}$  must be adjusted according to the slope in  $P_2$ . Plotting the required  $f_{xy}$  for an optimal positioning of  $a_1$  depending of the slope in  $P_2$  leads to the graph in figure 4.7. The hyperbolic graph could be expressed by the

Figure 4.7: Factor  $f_{xy}$  depending on the slope in  $P_2$ 

general form  $\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$ . A possible approach to determine the optimal  $f_{xy}$  would be fitting the hyperbole using an fitting algorithm such as least squares fitting (Weisstein, 2022). It would give a good fit, because a large set of points could be calculated, but it would require a major computing effort and depend on the project parameters. This

project specific relation could easily be calculated during the off-chain initialisation. The actual position of  $a_1$  within its bounds was analysed for an increasing slope in  $P_2$  while  $f_{xy}$  held constant. The example in figure 4.8, keeps the  $f_{xy}$ , that was calculated for an optimal positioning of  $a_1$  with a slope in  $P_2$  of  $1e - 10$ . Therefore the blue line (representing the actual  $a_1$  placement) crosses the green (optimal  $a_1$  placement) line. Further increasing the slope in  $P_2$  by keeping the calculated  $f_{xy}$  causes the actual  $a_1$  position to approach its upper bound, until it crosses the upper bound at  $1.8e - 10$ , violating the curvature conditions. Since  $f_{xy}$  is held constant, the slope in  $P_2$  can only

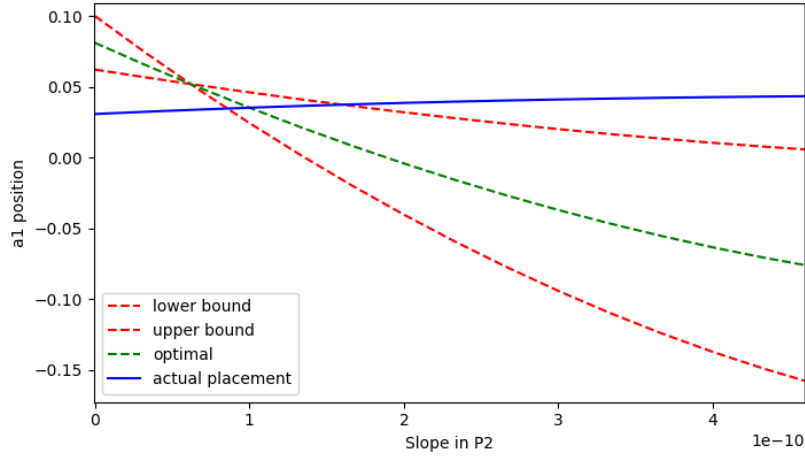
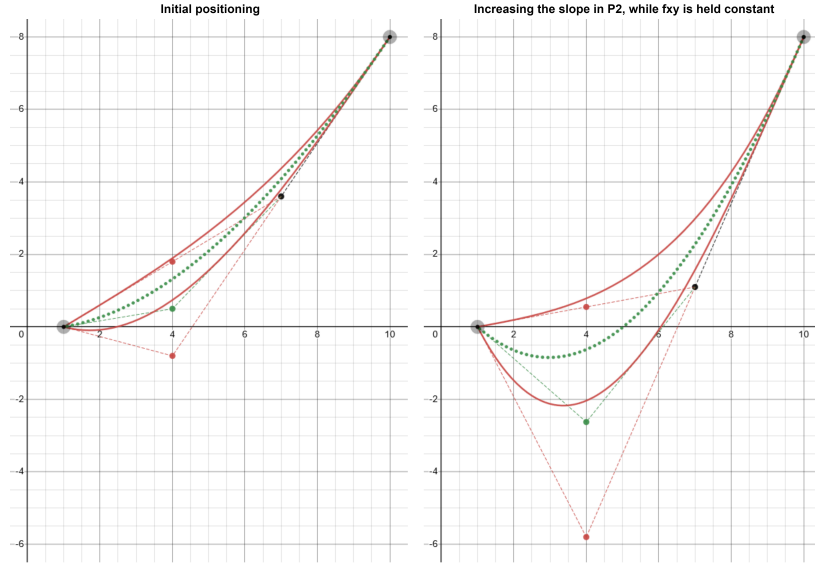
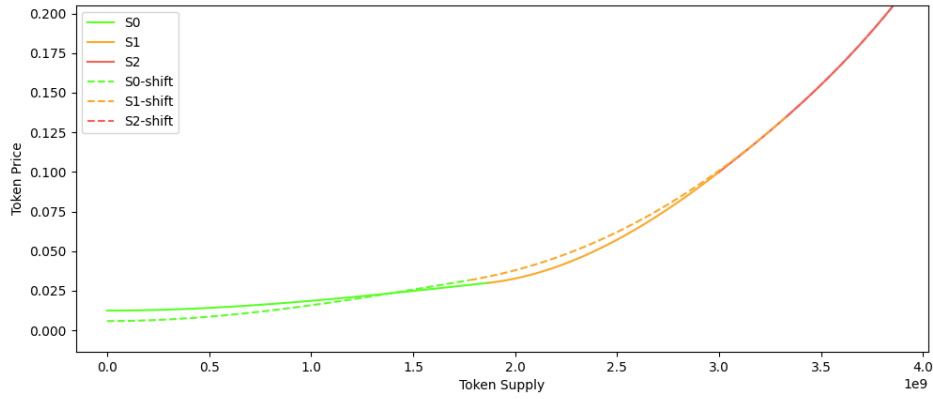


Figure 4.8:  $a_1$  placement in its bounds depending on the slope in  $P_2$

be increased by decreasing  $b_1$ . This directly influences the upper and lower bounds of  $a_1$ , because they depend on the slope in  $P_2$  and therefore the  $b_1$  placement. This effect is illustrated by actual Bézier curves in figure 4.9. The simplified form of Bézier curves used for the UBC is thus limited in its steepness, due to the positioning of  $b_1$  at  $t = \frac{2}{3}$ . Because the limitation is caused by the simplified form of Bézier curves, this approach holds  $f_{xy}$  constant. If  $a_1$  crosses its upper bounds, it will be set to the upper bound in order to comply to the curvature conditions. When shifting up  $P_2$ ,  $S_0$  and  $S_1$  must cross an additional  $\Delta Y$  and  $\Delta X$  in order to reach  $P_2$ . Since added  $\Delta Y$  and  $\Delta X$  are not necessarily in the same ratio, as the ratio defined by  $f_{xy}$ , this leads to unwanted effects on the curve shape. Figure 4.10 shows an example of a  $P_2$  shift from  $3e9$  to  $3.33e9$ . The curve shape shifts the pool balance to larger supply around  $2e9$  and decreases  $P_0$  vice versa. This effect is contrary to the concept of the UBC to constantly increase  $P_0$  and thus the minimum price for tokens. Therefore a new mechanism must be created to eliminate the decreasing  $P_0$  by shifting up  $P_2$ .

Figure 4.9: Effect of increasing the slope in  $P_2$  while  $f_{xy}$  is held constantFigure 4.10: Shifting up  $P_2$  while  $f_{xy}$  is held constant

#### 4.4.4 Flat region

In order to simplify shifting up  $P_2$ , by keeping  $f_y$ ,  $f_{xy}$  as well as keeping  $\Delta Y$  and  $\Delta X$  of  $S_0$  and  $S_1$  constant, a flat region left of  $S_0$  can be used. This flat region continues the zero slope in  $P_0$  at the height of  $P_0$  and shifts  $x_0$ ,  $x_1$  and  $x_2$  to the right. The length of the flat segment  $FS_0$ , corresponds to the total shift of  $x_2$  from its initial position. Therefore  $FS_0$  is initiated with a length of zero. The new  $FS_0$  must be included in the  $B_{pool}$  equation to fit its additional balance. Therefore equation 2.44 is adjusted by

subtracting the rectangle of FS0 from the pool balance to fit:

$$\begin{aligned}
 4B_{pool} - 4\Delta x_{flat}P_0 &= \Delta x_0 b_0 + \Delta x_0 2P_0 + \Delta x_1 [a_1 + b_1 + P_2] + (\Delta x_0 + \Delta x_1)P_1 \\
 4B_{pool} &= \left( \underbrace{4\Delta x_0 - \frac{3}{2}\Delta x_0 f_y - \frac{1}{2} \frac{\Delta x_1 \Delta x_1}{\Delta x_0} f_y}_{g_0} + \underbrace{2\Delta x_1 - 2\Delta x_1 f_y + 4\Delta x_{flat}}_{g_1} \right) P_0 \\
 &\quad - \left( -\frac{3}{2}\Delta x_0 f_y - \frac{1}{2} \frac{\Delta x_1 \Delta x_1}{\Delta x_0} f_y - 2\Delta x_1 f_y - \Delta x_1 \right) P_2 \\
 &\quad + \Delta x_1 b_1
 \end{aligned} \tag{4.66}$$

Shifting up  $P_2$  is done by shifting the whole segments S0 and S1 up and right. Therefore the  $\Delta Y$  and  $\Delta X$  of S0 and S1 do not increase. The basic process is illustrated in figure 4.11. Thus, adding two new areas to the pool balance when shifting up  $P_2$ . Both areas

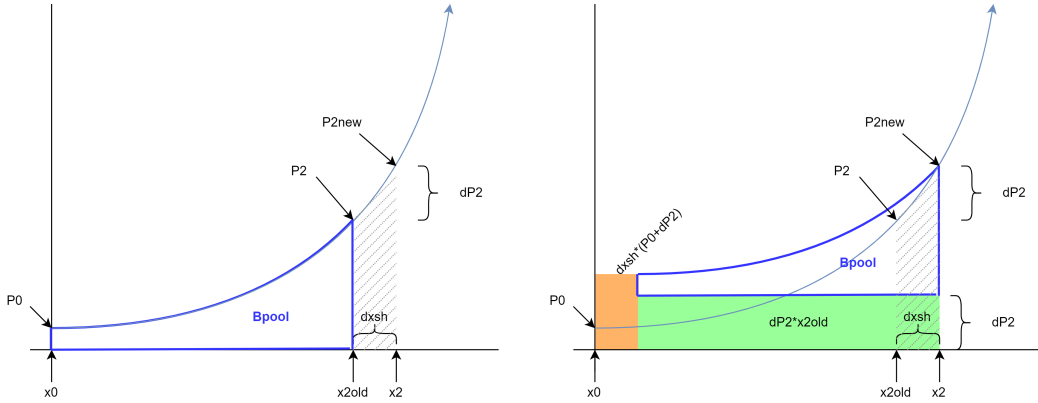


Figure 4.11: Concept of shifting up  $P_2$  with a flat region

are rectangles depending on the size of the actual up-shift, that can be expressed as:

$$\Delta P_2 = P_{2,new} - P_{2,old} \tag{4.67}$$

$$\Delta x_2 = x_{2,new} - x_{2,old} \tag{4.68}$$

Following this approach, the new pool balance includes the old pool balance  $B_{pool,old}$  (blue area), as well as the pool balance that is covered by the shift up itself  $\Delta B_{pool}$  (dotted grey area). Since the grey is insufficient for covering both rectangles, shifting

up  $P_2$  requires adding pool balance  $B_{pool,Add}$  as its done for undergirding the curve. Putting all together, a new equation for the pool balance can be expressed as:

$$\begin{aligned}
 B_{pool,new} &= B_{pool} + \underbrace{\Delta P_2 \cdot x_{2,old}}_{up-shift} + \underbrace{\Delta x_2 \cdot (P_0 + \Delta P_2)}_{right-shift} \\
 &= B_{pool,old} + \Delta B_{pool} + B_{pool,Add} \\
 B_{pool,old} + \Delta B_{pool} + B_{pool,Add} &= B_{pool} + \underbrace{\Delta P_2 \cdot x_{2,old}}_{up-shift} + \underbrace{\Delta x_2 \cdot (P_0 + \Delta P_2)}_{right-shift} \quad (4.69)
 \end{aligned}$$

The up-shift is controlled by the added pool balance  $B_{pool,Add}$  as the input parameter. Therefore the equation for  $\Delta x_2$  must be solved in order to determine the value for  $P_{2,new}$  and  $x_{2,new}$ . Because  $B_{pool,Add}$  depends on the actual shift  $\Delta x_2$ ,  $B_{pool,Add}$  is approximated by using a linear equation that undervalues the area due to the curvature conditions of the curve. Furthermore, a linear approximation for  $P_{2,new}$  is used in order to eliminate cyclic dependencies.

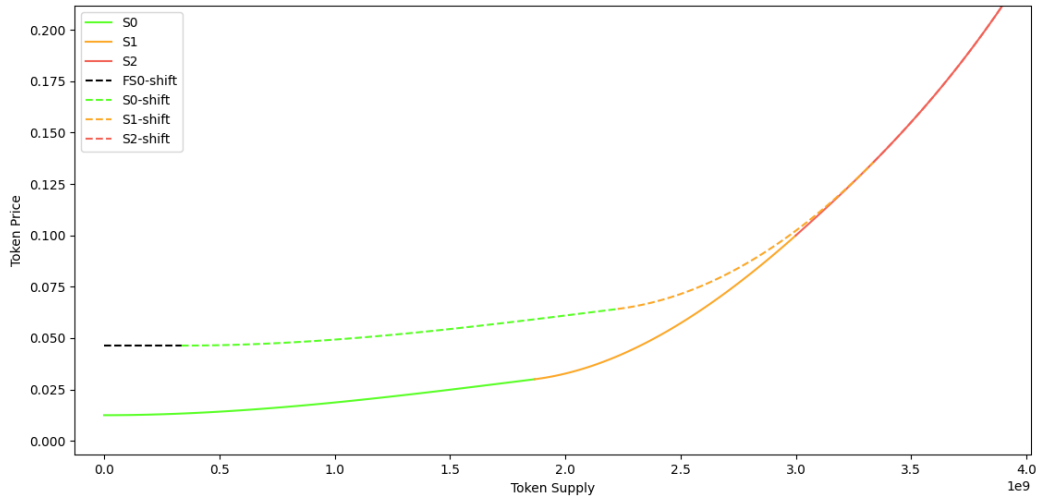
$$\Delta B_{pool,lin} = \frac{1}{2}(P_{2,new} - P_2)\Delta x_2 + P_2\Delta x_2 \quad (4.70)$$

$$P_{2,new} = \underbrace{\Gamma'_2(x = x_{2,old})}_k \Delta x_2 + P_2 \quad (4.71)$$

Inserting these linear approximations, equation 4.69 can be solved for  $\Delta x_2$ :

$$\begin{aligned}
 B_{pool,Add} &= -\Delta B_{pool} + (P_{2,new} - P_2)x_{2,old} + \Delta x_2(P_0 + P_{2,new} - P_2) \\
 B_{pool,Add} &= -\frac{1}{2}(k\Delta x_2 + P_2 - P_2)\Delta x_2 + P_2\Delta x_2 + (k\Delta x_2 + P_2 - P_2)x_{2,old} \\
 &\quad + \Delta x_2(P_0 + k\Delta x_2 + P_2 - P_2) \\
 \Delta x_{2,1/2} &= -\frac{(k \cdot x_{2,old} + P_0 - P_2)}{k} \pm \sqrt{\left(\frac{(k \cdot x_{2,old} + P_0 - P_2)}{k}\right)^2 + \frac{2}{k}B_{pool,Add}} \quad (4.72)
 \end{aligned}$$

Using the calculated  $\Delta x_2$ , the other parameters can be determined as well. The  $P_2$  shiftup is calculated by shifting  $x_0$ ,  $x_1$  and  $x_2$  by  $\Delta x_2$  to the right and shifting  $P_0$ ,  $P_1$  and  $P_2$  by  $\Delta P_2$  up. These new parameters are re-fitted to the actual pool balance, by running the fitting algorithm with the new pool balance and slope in  $P_2$ . Due to the approximation of  $B_{pool,Add}$ , a slight degirding occurs, since the actual pool balance that fits under the curve slightly decreased. An example of shifting up  $P_2$  is shown in figure 4.12. This examples also shifts  $P_2$  to 3.33e9 as it is illustrated in figure 4.10 executing the shiftup without a flat region.

Figure 4.12: Example - shifting up  $P_2$  with a flat region

#### 4.4.5 Quadratic Segment

Since the UBC should offer an open supply, the curve must be continued after S2 and  $P_3$ . As described by in section 2.4.1, there are two major options: (1) extrapolating S2 or (2) adding an open-ended S3.

Extrapolating S2 would mean using  $t > 1$  for the Bézier curve representation. If the curve would further increase, this would represent a suitable solution, since it would not add much complexity. Testing the extrapolation of S2 shows, that for some project parameters sets, the curve starts to bend down again resulting in a  $S$  shape. Figure 4.13 shows an example curve shape, that uses a slightly lower slope in  $P_3$ , resulting in a negative curvature. Thus, extrapolating S2 cannot be used for the UBC-MM. Using

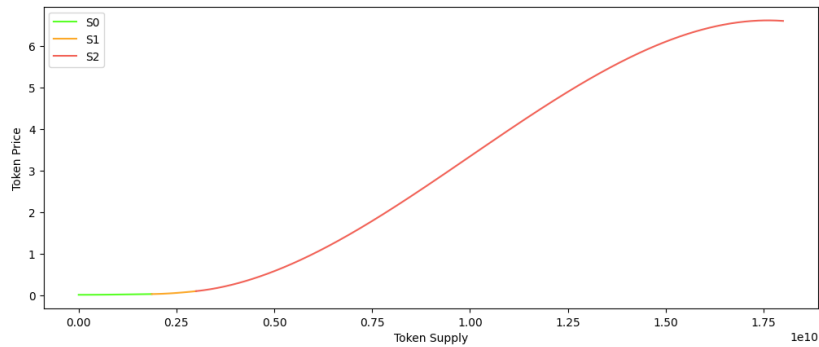


Figure 4.13: Example - Extrapolating S2

an additional Bézier curve as the open-ended S3 would lead to the same problem as extrapolating S2. Therefore this approach uses a quadratic segment. Using a quadratic

segment allows to ensure, that the slope is strictly increasing. The quadratic segment uses the general form of:

$$f(x) = ax^2 + bx + c \quad (4.73)$$

Therefore the three variables  $a$ ,  $b$  and  $c$  must be calculated in order to smoothly fit to S2. To calculate the variables, three characteristics of the quadratic curve must be given. First, the quadratic curve must cross the point  $P_3$ ,  $x_3$ . Second, the curve must continue the slope of S2 in  $x_3$ . Third, the curve must continue the curvature of S2 in  $x_3$ . These three characteristics can be expressed as:

$$f(x_3) = ax_3^2 + bx_3 + c = P_3 \quad (4.74)$$

$$f'(x_3) = 2ax_3 + b = \Gamma'_2(x_3) \quad (4.75)$$

$$f''(x_3) = 2a = \Gamma''_2(x_3) \quad (4.76)$$

These equations for a linear equation system, that can be solved as:

$$a = \frac{1}{2} \cdot \Gamma''_2(x_3) \quad (4.77)$$

$$b = \Gamma'_2(x_3) - 2ax_3 \quad (4.78)$$

$$c = P_3 - ax_3^2 - bx_3 \quad (4.79)$$

These parameters can be used to create the quadratic segment. An example of a fully initialized UBC is shown in figure 4.14. The quadratic segment smoothly fits to the right connection point  $P_3$  of S2 and continues its slope as well as curvature.

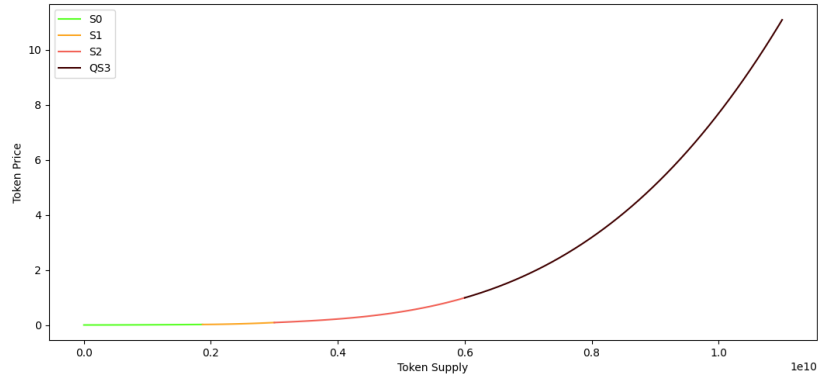


Figure 4.14: Example - Open-ended quadratic segment S3

### 4.4.6 Fitting Algorithm Evaluation

As argued before, the fitting algorithm derived within cycle three fulfils the curve shape requirements  $CS1$ ,  $CS2$  and  $CS3$ .

For undergirding actions, the  $CS4$  is fulfilled by definition of the undergirding event. Undergirding actions are only allowed, if the current trading point is beyond  $P_2$ . Since the undergirding action derived in this cycle adds pool balance and refits  $S0$  and  $S1$ ,  $S2$  is unchanged. Therefore the curve keeps its characteristics in the region of the trading point and beyond.

For  $P_2$  shiftup actions, the current trading point must be beyond the shifted  $P_2$  as well. During the shiftup process,  $S2$  must be reinitialized, because  $P_2$  is shifted and the slope in  $P_2$  is increased. The slope in  $P_3$  stays constant, whereas there are no changes in the upper region of the curve. In the lower region, the curve shape should have slight changes. Figure 4.15 shows the previous  $P_2$  shiftup as in figure 4.12, but zoomed on the  $P_2$  region.  $P_2$  gets shifted by  $0.33e9$ . The dotted blue line represents the new  $S2$  curve. As it can be seen in the figure, there is no visible changes divergence. A more detail analysis by calculating the maximum divergence of the new  $S2$  to the old  $S2$  has shown, that the maximum divergence is  $1.2e - 5$  resulting in a price change of  $0.00625\%$  at this point. Therefore,  $CS5$  would be sufficiently fulfilled by the approach derived within cycle three. It can be argued, that these minimal price changes are negligible in the presence of  $370\%$  price movement at  $P_0$ . Furthermore, this example  $P_2$  shiftup represent a major shiftup, which would not have been execute in a production environment. The approach derived within cycle three fulfils  $CS1$ ,  $CS2$ ,  $CS3$  and

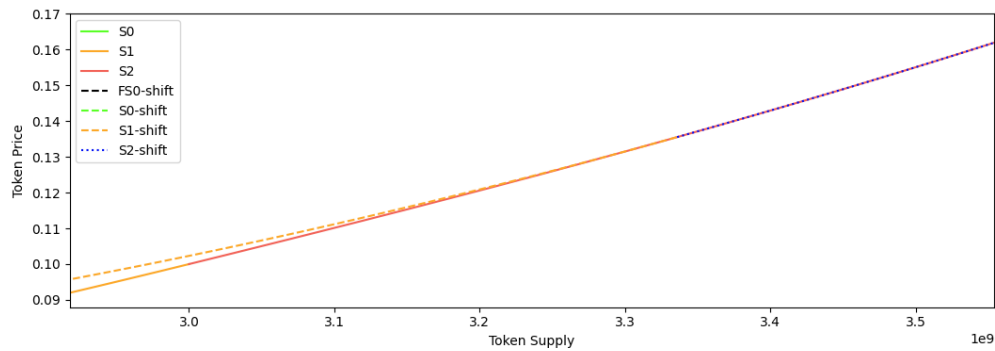


Figure 4.15: Example -  $P_2$  shiftup zoomed on  $S2$  changes

$CS4$ , while  $CS5$  is acceptably solved with negligible changes in  $S2$ . The first research question *RQ1: How can two cubic Bézier curves be injectively fitted to a fixed set of project parameters at given restrictions?* is answered by the derived approach. The

UBC-MM implementation in the next cycle therefore implements this approach in Cosmos and strives to answer the second research question.

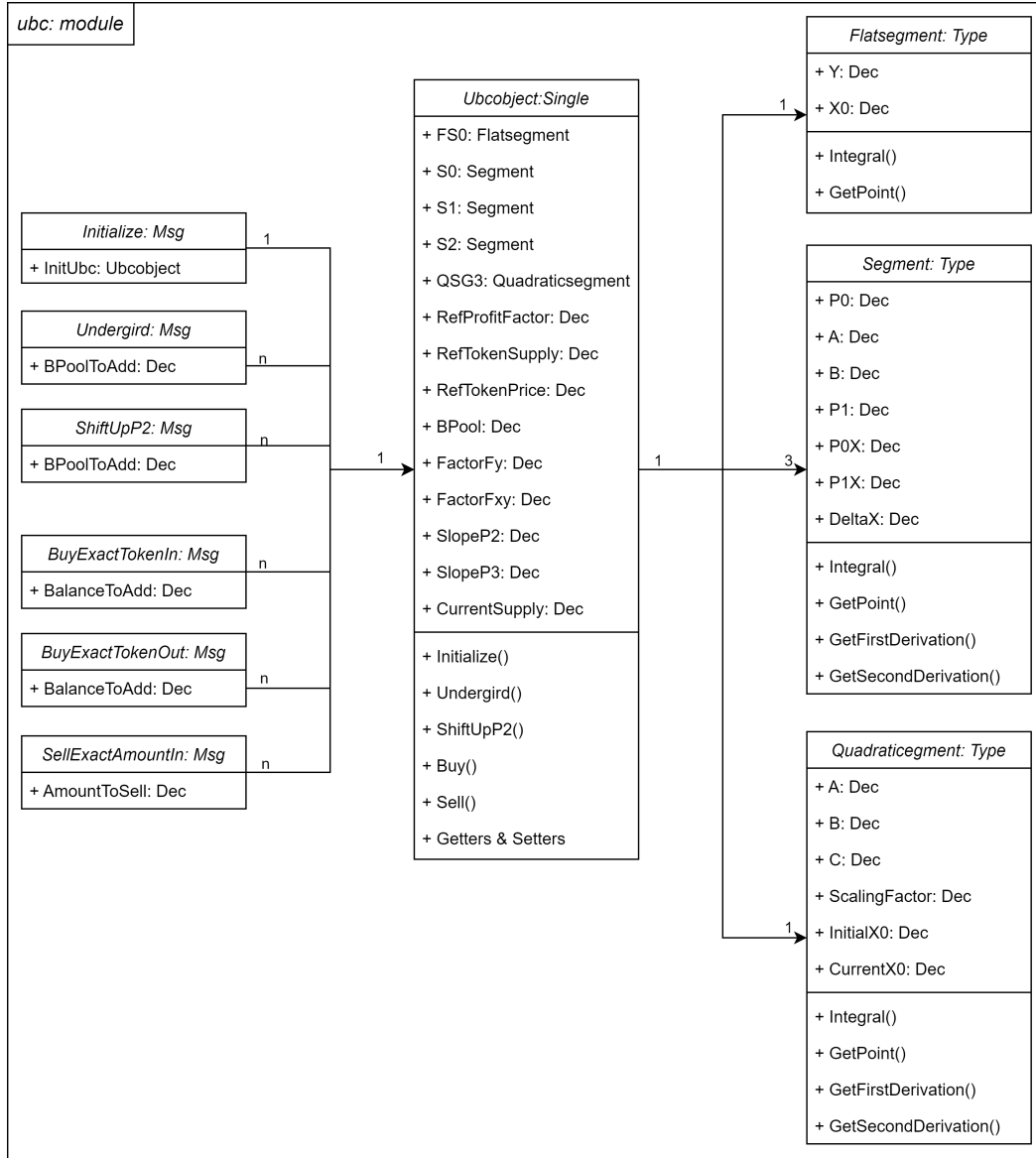
## 4.5 Cosmos Implementation

Cycle four focuses on the instantiation of the fitting approach derived in cycle three. To illustrate the most important aspects of the Cosmos implementation, UML diagrams and descriptions of the procedures are used in the following. These serve as an abstraction of the functionality. The sourcecode for the complete implementation is available in the appendix. The description of the prototype starts with the *ubc* module at the top level that gives an overview over the basic structure. Afterwards, the individual objects such as the persistent storage object, associated types and messages are described in detail. The integration to the bank module of the Cosmos SDK is discussed within the actual messages that implement the integration.

### 4.5.1 Basic Structure of the Prototype

The UBC prototype consists of four objects, that are used to represent the curve: (1) The *Ubcobject* that stores all information about the curve, being synchronized across the network, (2) the *FlatSegment* storing the parameters for FS0, (3) the *Segment* storing the parameters of S0, S1 and S2 individually and (4) the *QuadraticSegment* storing the parameters for the endless quadratic segment.

For the Cosmos implementation, all objects are included into a new *ubc* module. The *Ubcobject* is instantiated as a *single*, being a Create Read Update Delete (CRUD) object for data (Ignite Cli Docs, 2022a). The three segment objects are instantiated as *types* being referenced by the *Ubcobject*. To interact with the UBC, there are six messages assigned to the *ubc* module: (1) *Initialize* to bootstrap the curve, (2) *Undergird* to undergird the curve, (3) *ShiftUpP2* to execute a shift up, (4) *BuyExactAmountIn* to buy tokens for a given amount of input, (5) *BuyExactAmountOut* to buy an exact amount of tokens for a variable input and (6) *SellExactAmountIn* to sell an exact amount of token. The entire structure of the *ubc* module and the interconnections are visualized in the UML class diagram in figure 4.16.

Figure 4.16: UML class diagram of the *ubc* module

### 4.5.2 Ubc Object

The *Ubcobject* is represented by a *single*, that stores all information of the UBC. The *single* data type is used to persist the data and exchange it with the nodes via the network layer of Tendermint. The other objects in contrast are represented by types, that are not persistent on their own. References on actual instantiations of these types are stored within the *Ubcobject* in order to be persistent. Furthermore, the *Ubcobject* stores the project parameters such as the reference profit factor, reference

token supply, reference token price, initial pool balance and slope in  $P_2$  and  $P_3$ . In addition, the *Ubcobject* stores the current supply for the execution of trade actions. All these parameters are represented by decimals. The *decimal* data type and its effect on the UBC calculations is further described in section 4.5.8.

The *Ubcobject* implements the methods uses to interact with the UBC. These methods implement the actual actions for the UBC such as: (1) initialization of the curve, (2) undergirding the curve, (3) shifting up  $P_2$ , (4) buying tokens and (5) selling tokens. In addition getter and setter methods are used to mask the interaction with the actual segments to prevent false updates.

### 4.5.3 Flat Segment

The *Flatsegment* is a *type* in Cosmos that stores the  $Y$  and  $X0$  values that should be equal to  $P_0$  and  $x_0$ . In order to ensure their equality, the setter for  $P_0$  in the *Ubcobject* sets the value in FS0 as well as in S0 and vice versa for  $x_0$ . It implements the *Integral* to get the area under the segment with the given upper and lower bound. The *GetPoint* method returns the  $y$  value at a given input parameter  $x$ . Since the UBC only uses one flat segment, there is a 1:1 connection to the *Ubcobject*.

### 4.5.4 Segment

The *Segment* is a *type* in Cosmos that stores the  $y$ -value of the control points  $P_0$ ,  $a$ ,  $b$  and  $P_1$  used for the Bézier curve. Since the UBC only uses the  $y(t)$  form of Bézier curves, the *Segment* object also stores the  $x$ -values of the outer control points  $P_0$  and  $P_i$ . In order to simplify the further calculations, the  $\Delta x$  is stored as well. The *Segment* object implements the mathematical functions of the Bézier curve such as: (1) *GetPoint* being the implementation for  $\Gamma(t)$ , (2) *GetFirstDerivation* for  $\Gamma'(t)$ , (3) *GetSecondDerivation* for  $\Gamma''(t)$  and (4) *Integral* for  $\int \Gamma(t)dt$ . The actual mathematical equations of Bézier curves are defined in section 2.4.2.

### 4.5.5 Quadratic Segment

The *Quadraticsegment* is a *type* in Cosmos that stores the parameters for the quadratic function that represents the endless segment S3 for the UBC. Therefore it stores the

variables  $A$ ,  $B$  and  $C$ . Since the x-axis operates on much larger values than the y-axis, a scaling factor is used to avoid extremely small slopes. Practical experiments have shown that without a scaling factor the slope can no longer be represented with sufficient accuracy by the *decimal* data type due to the limited precision. In order to enable larger undergirding and  $P_2$  shift-ups, the *Quadraticsegment* stores its initial starting  $x$ -value as well as the future  $x$ -value in the case of an potential shifting of  $P_3$ . Furthermore, it implements the same mathematical function as the *Segment*.

### 4.5.6 Bank Integration

The *Ubcobject* represents the persistent status of the UBC. In the absence of undergirding and shiftup-events, the only value changing in *Ubcobject* is the current supply. To ensure the interoperability of the tokens issued by the UBC with other modules and chains, the UBC is integrated in the *bank* module of the Cosmos SDK. Therefore the *ubc* module has full control over the project tokens. It uses the bank modules keepers to mint and burn tokens according to the operation of the UBC. Buying tokens against the UBC will mint new tokens that are send to the buyer and burn tokens vice versa when selling them. Therefore the *ubc* module itself never holds project tokens. Instead it holds the euro voucher token used to back the UBC. The amount of euro voucher tokens held by the UBC is increased in the case of buy actions and decreased for sell actions vice versa.

The *bank* module handles all other functionalities of the token such as sending tokens, using tokens to buy services or paying gas fees.

The project token (CAX) and euro voucher token (EVCH) are represented by coins in the Cosmos SDK. Both use an exponent of 6 with their base currency being *ucax* and *uevch*. They are defined in the *config.yml* that is used by the Ignite CLI, to create the coins. An example configuration is shown in the listing below:

---

```
- description: "The CaX Coin"
  display: "cax"
  base: "ucax"
  name: "Catena-X Coin"
  symbol: "CAX"
  denom_units:
    - denom: "ucax"
      exponent: 0
```

```
- denom: "cax"
  exponent: 6
```

---

### 4.5.7 Messages

As discussed in section 2.1, the Cosmos SDK routes messages are routed to their individual modules. Therefore the *ubc* module implements its own messages in order to interact with the UBC. These messages are described in the following.

#### Initialize

The *Initialize* messages handles the initialization of the UBC and therefore is in a 1:1 relation to the *Ubcobject*. Within the message routine, the initial supply of project tokens is minted and distributed according to the initial distribution plan. In order to give initial funding to the UBC, the sender of the message must provide the entire initial pool balance in form of euro voucher token. The message routine executes the initial fit of the *Ubcobject* according to the created algorithm in cycle three. The full parameter sweep as described in cycle three to determine  $f_{xy}$  for an optimal  $a_1$  placement must be calculated off-chain.  $f_{xy}$  must therefore be provided in the initialization process. Experiments during the development of the prototype have shown, that the full parameter sweep causes *broadcastTxCommit* errors, due to the runtime of the parameter sweep.

#### Undergird

The *Undergird* message handles the undergirding of the UBC and therefore is in a n:1 relation to the *Ubcobject*. The message routine calculates the difference of the pool balance in the *Ubcobject* and the actual euro voucher token holdings of the *ubc* module. The difference, that was previously collected by trading fees, can be used to undergird the curve.

**ShiftUpP2**

The *ShiftUpP2* messages handles the shiftup of  $P_2$  in the UBC object. It therefore has a n:1 relation to the *Ubcobject*. The message routine executes the shiftup of  $P_2$  in the *Ubcobject* as described in cycle three. Furthermore it checks if the euro voucher token holdings of the module are sufficient to add new pool balance to the curve.

**BuyExactTokenIn**

The *BuyExactTokenIn* message handles buying tokens against the UBC with a given input amount in euro voucher tokens. It is mapped in a n:1 relation to the UBC. The implementation of the message uses the Taylor expansion in order to calculate the amount of project tokens, that can be bought for the given input amount. Since the Taylor expansion only approximates the amount of tokens used, the implementation uses recursion, to shrink the error under a defined threshold. The message transfers the amount of euro voucher token specified as input to the module's account and mints new project tokens, which are then transferred to the buyer in exchange.

**BuyExactTokenOut**

The *BuyExactTokenOut* also handles buying tokens against the UBC with a n:1 relation. In contrast to the *BuyExactTokenIn* message, this message has a given amount of output tokens nominated in project tokens. This amount is calculated by the integral as discussed before in section 2.4.8. The bought tokens are newly minted by the module and transferred to the buyer.

**SellExactAmountIn**

The *SellExactAmountIn* handles selling tokens against the UBC with a given amount to sell. It is mapped in a n:1 relation to the UBC. The message routine implements the selling approach as described in section 2.4.8. After calculating the amount of pool balance, that is crossed by selling the project tokens, the seller is charged by this amount of project tokens. The module then burns these project tokens and sends the corresponding amount of euro voucher token to the seller.

### 4.5.8 Decimal Representation

In order to handle decimals in a distributed system, the Cosmos SDK uses a custom *decimals* data type (Cosmos SDK, 2022a). The *decimals* data type is based on big integers, which are implemented in the golang module *math/big*. These big integers could represent infinitely large numbers, since they are based on a sliced number representation (Golang Docs, 2022a, 2022b). However the size of big integers in the Cosmos SDK is limited to 256 bits by a parameter (Cosmos SDK, 2022b). The *decimal* data type inherits this limitation representing the integer part of the decimal. Additionally, the *decimal* data type adds a precision, which is by default 18 digits. These 18 digits translate to 60 bits, that are added to the maximum bit length of the *decimal* (Cosmos SDK, 2022a). Thus, the Cosmos SDK scales the integers by  $10^{\text{precision}}$  in order to calculate decimals. This precision is also defined as parameter and could be adjusted by forking the math library.

The UBC-MM uses the *decimals* data type for calculation. Since *decimals* have a limited precision, they are truncated after this configured precision. The Cosmos implementation of *decimals* truncates the intermediate result after each arithmetic operation, leading to possible inaccuracies (Cosmos SDK, 2022a). Therefore small intermediate results could affect the end result massively. Assuming there is a 18 digit precision, decimals smaller than  $e-12$  can reduce the accuracy of all results to 6 digits. An example of scaling  $\pi$  using the *decimal* data type is shown in equation 4.80.

$$\begin{aligned}\frac{\pi}{1e12} &= 0.000000000000314159 \\ \frac{\pi}{1e12} \cdot 1e24 &= 314159000000.0\end{aligned}\tag{4.80}$$

Regarding the token, that often operate with supplies of up to  $1e12$  by some tokens, decimal smaller than  $1e6$ , that are scaled back and forth, could already cause an inaccuracy of one token.

Using tokens with an exponent of 6, this potential inaccuracy could not be seen during first tests of the UBC-MM. However, it should be kept in mind for the later evaluation of the numerical accuracy of the UBC-MM.

# 5 Evaluation

Following the research approach as described in section 3, the UBC-MM using the newly created fitting algorithm is evaluated within this evaluation section. The evaluation is structured according to the set requirements as shown in table 3.2. The requirements are evaluated by simple use case test as well as simulations using the CLI interface to execute on-chain transactions, if applicable. The examples use the denom *ucax* as the project token and *uevch* as the euro voucher token. The requirements are split in functional and system requirements.

## 5.1 Functional requirements

The functional requirements describe functionalities, that the UBC-MM must provide in order to work as intended. Referring to the research method section 3, the functional requirements are as follows:

|            | No.        | Requirement                                                                         | fulfilled |
|------------|------------|-------------------------------------------------------------------------------------|-----------|
| functional | <b>MM1</b> | The UBC-MM enables users to buy project tokens in exchange for euro voucher token.  | yes       |
|            | <b>MM2</b> | The UBC-MM enables users to sell project tokens in exchange for euro voucher token. | yes       |
|            | <b>MM3</b> | The UBC-MM extracts a percentage trading fee.                                       | yes       |

Table 5.1: Requirements for the UBC-MM implementation - functional

### [MM1] buy project tokens

The UBC-MM enables users to buy project tokens in exchange for euro voucher token. The users has two choices in order to buy project tokens: (1) providing an input amount in euro voucher token or (2) providing the desired output amount of project token.

If the user chooses to provide an input amount in euro voucher token, the UBC-MM

uses an iterated Taylor approximation in order to determine the amount of project token. Otherwise, the UBC-MM calculates the integral starting at the current supply up to the current supply plus the wanted output amount of project token resulting in the area under the curve and therefore the amount of euro voucher token.

Buy transactions can be executed in the CLI using the compiled go binary. CLI commands for transactions follow the same structure for all UBC interactions. An example transaction buying 1000000 project tokens would be executed by:

```
> ubcd tx ubc buy 1000000 -from alice -y
```

This example buy transactions results in *alice* buying 997000 *ucax* token in exchange for 1000000 *uevch*. The effective token price is 0.1 *uevch/ucax*, while the buy transaction is charged by a trading fee of 0.3%, that is subtracted. The listing below shows the token balances reported by the bank module via an API call.

**before:**

```
{"balances": [
{"denom": "ucax",
"amount": "3000000000000000"},
{"denom": "uevch",
"amount": "1000000000000000"}]}
```

**after:**

```
{"balances": [
{"denom": "ucax",
"amount": "3000000009970000"},
{"denom": "uevch",
"amount": "999999990000000"}]}
```

## [MM2] sell project tokens

The UBC-MM enables users to sell project tokens in exchange for euro voucher token. The user gives the amount of project tokens to sell as an input, whereas the UBC-MM calculates the corresponding euro voucher tokens, that are freed and subtracts the trading fee. The example below illustrates Alice selling the 997,000 *ucax*, she bought in the buy transaction above. The loss of 5,991 *uevch* is caused by the trading fee, that charges 0.3% twice.

**before:**

```
{"balances": [
{"denom": "ucax",
"amount": "3000000009970000"},
{"denom": "uevch",
"amount": "999999990000000"}]}
```

**after:**

```
{"balances": [
{"denom": "ucax",
"amount": "3000000000000000"},
{"denom": "uevch",
"amount": "9999999994009"}]}
```

**[MM3] extracting a percentage trading fee**

The UBC-MM extracts a percentage trading fee of 0.3% on all trades. The fee is always charged in euro voucher tokens in order to accumulate euro voucher tokens, that can later be used to undergird the curve. The working trading fee subtracting has been shown in the two examples of *MM1* and *MM3* above.

**5.2 System requirements**

The system requirements must be met to ensure that the UBC-MM functions correctly on a continuous basis. Referring to the research method section 3, the system requirements are as follows:

|        | No.        | Requirement                                                                             | fulfilled |
|--------|------------|-----------------------------------------------------------------------------------------|-----------|
| system | <b>MM4</b> | The UBC-MM is numerically accurate for all interactions.                                | yes       |
|        | <b>MM5</b> | The UBC-MM integrates with the standard bank module of the Cosmos SDK.                  | yes       |
|        | <b>MM6</b> | The UBC-MM fulfils the curve shape requirements, especially <i>CS4</i> and <i>CS5</i> . | yes       |
|        | <b>MM7</b> | The UBC-MM rejects invalid curve shapes and only operates on valid curve shapes.        | yes       |

Table 5.2: Requirements for the UBC-MM implementation - system

**[MM4] numerical accuracy**

The numerical accuracy and efficiency<sup>1</sup> of the UBC-MM is one of the most crucial requirements in order to use the UBC-MM in a productive environment. Within the scope of this work, the numerical accuracy will be evaluated by trading simulations and invariance tests. Multiple buy and sell actions are simulated using a python script, that triggers the transaction from the CLI, to simulate real users buying or selling tokens. The python script calculates the expected result parallelly to the UBC in order to check, if the result matches the actual result calculated by the UBC-MM. Since the

<sup>1</sup>hence e.g. no exponential functions (see section 2.4 or large number scaling (see section 4.5.8))

python script uses 64 bit floating point numbers for the calculation, they offer sufficient accuracy to check if the limited decimal representation of the UBC-MM causes numerical inaccuracies.

### Test case 1

*DESCRIPTION:* Alice buys 100 *CAX* tokens and sells the received *CAX* tokens afterwards.

*EXPECTED RESULT:* Alice must pay trading fees, therefore the amount gets reduced by 0.3% for each interaction. In the absence of trading fees, selling the tokens should exactly compensate buying the tokens. Therefore Alice's amount should be reduced only by the trading fee. This can be calculated as follows:  $loss = 100 \cdot 0.003 + 100 \cdot 0.003 = 0.6CAX$ . Applying the spot price of 0.1 *EVCH/CAX*, this results in a loss of 0.06 *EVCH* or 60,000 *uevch*.

*ACTUAL RESULT:*

---

```
Start balance of alice:
| 3000000000000000000 ucax
| 100000000000000000 uevch
End balance of alice:
| 3000000000000000000 ucax
| 99999999939999 uevch
```

---

After buying and selling the tokens, Alice has a loss of 60,001 *uevch* being equal to 0.060001 *EVCH*. This result matches the expected result with a divergence of 1 *uevch*. Analysing the code, the divergence of 1 *uevch* results of ceiling decimal result to the precision of *uevch*. Increasing the exponent of the euro voucher token would therefore reduce the divergence. Furthermore, the divergence is negligible, since it refers to a euro value of 0.000001 euro.

### Test case 2

*DESCRIPTION:* Alice buys 1,000 *CAX* tokens 10 times.

*EXPECTED RESULT:* Buying 1,000 *CAX* 10 times should lead to the same result as buying 10,000 *CAX* once. Therefore results should match.

**ACTUAL RESULT:**


---

```

Buying 1000 CAX 10 times
| 300001000000000000 ucax
| 99998996994980 uevch
Buying 10000 CAX once
| 300001000000000000 ucax
| 99998996994984 uevch

```

---

The test shows a divergence of  $-4$  *ucax*, when Alice buys 10 times instead of once. This divergence is caused by the rounding (ceiling) during trading fee calculation. As argued before this divergence is negligible.

**Test case 3**

**DESCRIPTION:** Alice sells the entire *CAX* supply at the initial trading point and buys the same amount again.

**EXPECTED RESULT:** Since the UBC is initialized with a pool balance of  $100e6$ , Alice should get this amount of *EVCH* minus the trading fees. The *Ubcobject* should also show a *currentSupply* and *BPool* of exactly zero. Buying the tokens again should cost Alice  $100e6$  *EVCH* and result in the initial values for the *currentSupply* and *BPool*.

**ACTUAL RESULT:**

| Balances                 | Ubcobject                          |
|--------------------------|------------------------------------|
| Start balance of alice:  | Initial Ubcobject:                 |
| 300000000000000000 ucax  | currentSupply: 3000000000.00000... |
| 100000000000000000 uevch | BPool: 100000000.00000...          |
| Balance after selling:   | Ubcobject after selling:           |
| None ucax                | currentSupply: 0.00000...          |
| 199700000105266 uevch    | BPool: -0.105583000676431076       |
| Balance after buying:    | Ubcobject after buying:            |
| 300000000000000000 ucax  | currentSupply: 3000000000.00000... |
| 99399999999366 uevch     | BPool: 100000000.00000...          |

Alice sells the total supply of  $3e9$  *CAX* and gets 99,700,000.105266 *EVCH* in return. Subtracting the trading fee of the initial pool balance, Alice gets 0.105266 *EVCH* too much. These *EVCH* paid too much are also represented in the *Ubcobject* having a

slight negative value for the *BPool* value. Buying the total supply of  $3e9$  *CAX* again brings the *Ubcoject* exactly back to its initial values. The balance after buying the tokens matches the expected result subtracting the trading fee twice. There is still a divergence of  $0.000634$  *EVCH*.

Compared to the transaction volume of  $100e6$  euro, the divergence causes by the inaccuracies are  $0.105$  euro resulting in a loss of  $0.0000000105\%$  for Alice. Regarding the magnitude of the potential loss of Alice, it can be argued that the inaccuracies causing a loss of  $0.0000000105\%$  are negligible in the presence of  $0.3\%$  trading fees. The test case has shown, that minor numerical inaccuracies occur, when executing buy and sell transaction in edge cases. However, these numerical inaccuracies compensate each other. Therefore, it could be argued that these inaccuracies are caused by the initial fit. This results in the next test case 4.

#### Test case 4

*DESCRIPTION:* To further trace down the deviation of test case 3, the UBC-MM is initialized with  $100,000,000$  *EVCH*. To test, if the pool balance matches the amount of initialized *EVCH*, the curve is integrated.

*EXPECTED RESULT:* Since the UBC-MM is initialized with  $100,000,000$  *EVCH*, the pool balance of the *Ubject* should be  $100,000,000$  as well.

*ACTUAL RESULT:*

---

```
Integral 0 to 3e9
| 100000000.105583000676431076
Integral 3e9 to 0
| -100000000.105583000676431076
```

---

Both integrals show the same divergence of  $0.105583000676431076$ , resulting into an inaccuracy of  $0.1$  euro. Since executing the same calculation with floating point numbers in python lead to the same divergence, it can be argued, that the difference is caused by inaccuracies during the initial fit. Regarding the amount of initialized *EVCH*, the deviation corresponds to  $0.0000001\%$ . Thus, it can be argued that the deviation is negligible. Only the  $0.1$  euro would have to be added to the pool balance and *EVCH* amount, in order to have an exact initialization.

**Test case 5**

*DESCRIPTION:* Alice sells the entire *CAX* supply at the initial trading point and buys the same amount again. The pool balance correction of test case 4 is applied during the initialization.

*EXPECTED RESULT:* Alice should get the full pool balance of 100,000,000.105..

*EVCH* and the pool balance variable of the *Ubcoject* should not be negative.

*ACTUAL RESULT:*

| Balances                  | Ubcoject                           |
|---------------------------|------------------------------------|
| Start balance of alice:   | Initial Ubcoject:                  |
| 3000000000000000000 ucax  | currentSupply: 3000000000.00000..  |
| 1000000000000000000 uevch | BPool: 100000000.10558...          |
| Balance after selling:    | Ubcoject after selling:            |
| None ucax                 | currentSupply: 0.00000...          |
| 199700000105266 uevch     | BPool: 0.00000...                  |
| Balance after buying:     | Ubcoject after buying:             |
| 3000000000000000000 ucax  | currentSupply: 3000000000.00000... |
| 993999999999366 uevch     | BPool: 100000000.10558...          |

Alice sells the total supply of  $3e9$  and get 99,700,000.105266 *EVCH* in return. The *Ubcoject* has a *BPool* value of exactly zero. After Alice buys the tokens again, the *BPool* value moves back to the corrected pool balance. Selling and buying the total supply is now working fine the minimal adjustment of test case 4.

**[MM5] bank module integration**

The UBC-MM is integrated into the bank module of the Cosmos SDK. The UBC-MM prototype implemented within the scope of this work uses two token: (1) *CAX* representing the project token and (2) *EVCH* representing the euro voucher token. Both tokens use an exponent of 6. Therefore their base token is *ucax* and *uevch*. The UBC-MM also uses the bank modules native keepers in order to mint and burn *ucax*. The working integration is shown in the evaluation of *MM1* and *MM2*, since the before and after states queried the API of the bank module with *Alice's* address.

**[MM6] curve shape**

The UBC-MM uses the final fitting algorithm derived within cycle three of the artefact description. The fitting algorithm itself fulfils all curve shape requirements. As shown in the evaluation of *MM4*, the numerical precision is sufficient for fitting the curve with the new fitting algorithm.

**[MM7] valid curve shapes**

The UBC-MM rejects invalid curve shapes by executing a full check on all curvature conditions for all UBC interactions, that could possibly lead to invalid curve shapes. If one or more curvature conditions fail, the transaction changing the curve shape gets aborted and the changes are rolled back.

The functionality was tested using insufficient pool balances for initializing the curve, too much pool balance for undergirding and too large shift ups of  $P_2$ . The UBC-MM aborted the transaction in all three test cases and continued using the old curve or no curve in the case of the initialization.

# 6 Discussion

In order to follow the publication schema of Gregor and Hevner et al. (2004), this section discusses the finding of this work, learnings, contributions to knowledge, limitations and further research aspects of the UBC-MM.

## 6.1 Curve shape fitting algorithm

This work derived a fitting algorithm for the UBC, that overcomes the limitations of the initial fitting algorithm (Poddey & Lamberty, 2022). The fitting algorithm fulfils all the requirements previously defined in the research method section 3. Thus, it produces curve shapes, that are (1) injective for a given set of project parameters, (2) fulfil their curvature conditions, (3) can be fitted without brute force scans, (4) enable undergirding and (5)  $P_2$  shiftups without changing the region of the current trading point. In order to fulfil these requirements, the fitting algorithm uses the factor  $f_y$  as introduced in the initial approach and replaces the factor  $f_x$  with a new factor  $f_{xy}$ . This new factor  $f_{xy}$  is used to store the x to y ratio of S1. This ratio is kept constant, since scaling S1 by keeping  $f_{xy}$  also keeps  $a_1$  positioned optimally. The optimal positioning of  $a_1$  is determined by its curvature conditions and initially calculated by an off-chain full parameter scan. Shifting up  $P_2$  is realized by introducing a flat segment, that allows to shift S0 and S1 up and right, without changing the curve shape. Only the slope in  $P_2$  is adjusted, while the positioning of  $a_1$  is kept.

Despite the successful improvements and learnings of the UBC-MM, there are still some limitations regarding the fitting algorithm. As shown, fitting, refitting, undergirding and shifting up  $P_2$  works as intended. The iterative development of the fitting algorithm led to a deeper understanding of the optimal curve shape, curvature conditions and behaviour of factor  $f_{xy}$ . Furthermore, it led to a new approach for fitting the curve by scaling both S0 and S1, while all their control points are optimally placed. However, this itself was impractical, but lead to further investigations on the optimal curve shape with only one control point placed optimally.

However, the first research question **RQ1**: *How can two cubic Bézier curves be injectively fitted to a fixed set of project parameters at given restrictions?* is answered

by the fitting algorithm derived within the scope of this work. It can injectively fit two cubic Bézier curves to a given set project parameters and simultaneously fulfil the defined curvature conditions.

## 6.2 UBC-MM Implementation

The implementation of the UBC-MM predominately focused on transferring the gained knowledge and mathematical equations into code. The final artefact fulfils all of its requirements as well. It therefore allows to: (1) buy project tokens, (2) sell project tokens, and (3) extract a percentage trading fee. In order to ensure a proper functioning, the UBC-MM is (4) numerically accurate for all interactions, (5) integrated into the standard bank module of the Cosmos SDK, (6) always fulfilling the curve shape requirements and (7) rejecting invalid curve shapes. In order to fulfil all requirements, the mathematical concepts of the UBC-MM were implemented in Cosmos SDK as a layer 1 blockchain. Since all interactions and functions were evaluated beforehand, the evaluation of the UBC-MM focused on the numerical accuracy. Thus, it could be learned, that the *decimal* data type offers sufficient accuracy, that even in edge cases the inaccuracy was magnitudes smaller than the charged trading fee.

Since the UBC-MM relies on the mathematical concept, the limitations of the fitting algorithm also apply for the UBC-MM. Furthermore, the security of the UBC-MM implementation has not been evaluated within the scope of this work. However, it will be extensively analysed within the context of further research activities at Bosch.

The second research question **RQ2:** *Can the UBC-MM be implemented using Cosmos SDK as a layer 1 blockchain, with adequate numerical accuracy?* can therefore be answered with yes, based on the above argumentation.

## 6.3 Further Research

The presented improvements and instantiation of the UBC-MM have shown the feasibility as well as known limitations. The limitations of the fitting algorithm are open for further research in order to find a solution on how to shift  $P_2$  beyond  $P_3$  or also shifting  $P_3$ . Furthermore, the in cycle two developed fitting algorithm could be further researched in order to find a practical algorithm, that utilizes the basic concept.

The UBC-MM opens a wide field of further research. First of all the initial considerations, that the UBC-MM can efficiently reduce the volatility and risk of market manipulations, could be researched in order to provide evidence for the assumptions of this work. An quantitative analytical research based on simulations should be suitable here.

In addition, there are further ideas regarding the UBC-MM that could further improve its usability and security. In the current state, all interactions are executed on a transaction level, which means that all interactions are directly execute against the curve with a *first come, first serve* logic as it is done for other AMMs as well. In order to reduce frontrunning and miner extractable value, a block-execution approach would be open for further research. Using a block-execution would ensure, that all traders within the same block get the same effective price, eliminating intra-block frontrunning. Furthermore, it could enable the UBC-MM to use an order book approach, in which buy and sell orders are directly matched and the outstanding buy or sell amount is traded against the curve. This approach would enable users to place limit orders, while the UBC-MM remains in control of the price since it is mapped to the underlying bonding curve. The token price would therefore only rise, if the buy orders outweigh the sell orders, while the price increment is then given by the slope of the bonding curve.

Another research direction to further decrease the risk or market manipulation especially *pump and dump* schemes could be a dynamic bid / ask spread for the UBC-MM. The bid / ask spread could be based on the recent trading volume, being elastically increased proportional to the volume. Thus, pumping the price up, would get more expensive, since the real price would be influenced less. Dumping the price down again, would also result in bigger losses for the manipulator, since the tokens are sold for less than the real price.

## 7 Conclusion

This work aims to instantiate a new form of bonding curve market maker, that solves liquidity, volatility and market manipulation issues in cryptoeconomic systems. Extracted value by the market maker stay within the system and is redistributed to late sellers. This work contributes to research in several ways. First, it holistically describes and derives the Undergirding Bonding Curve Market Maker (UBC-MM) for the first time based on the tokenization notebook of Poddey and Lamberty (2022). Second, it improves the fitting algorithm initially described in the tokenization notebook in order to bring the UBC-MM into production. The improved fitting algorithm initializes an optimal curve shape of the used Bézier curves and keeps this optimal curve shape by scaling the curve through a newly defined factor. It allows to bend up the curve by undergirding previously extracted value in order to be distributed to late sellers. Third, this work instantiated the UBC-MM as part of a layer 1 blockchain using the Cosmos SDK. It evaluated and analysed the functional as well as systematic requirements for productive usage. The numeric accuracy of the transferred mathematical concept using the native fixed precision *decimal* representation of irrational numbers was evaluated in detail, which led to the conclusion of a sufficient accuracy. Therefore this paper provided a first proof of concept for a possible productive implementation of the UBC-MM as part of the Catena-X ecosystem.

Finally, the UBC-MM could be used for Catena-X in order to create a low-volatile token, which lowers the entry hurdle into the Catena-X ecosystem. The risk of market manipulations could be reduced efficiently by the liquidity dynamics of the UBC-MM. Furthermore, the UBC-MM would redistribute extracted value to all participants in the ecosystem, continuously increasing the minimum token price. Therefore this work contributed to the cryptoeconomic system within Catena-X by instantiating a first prototype of the UBC-MM and further improving its fitting mechanism.

# A Appendix: Scaling approach

The full derivation of the scaling approach can be seen in the file *100\_scaling\_approach.lyx* included on the attached disk. The file can be opened using a LyX editor.

An alternative (cropped) PDF export is included in the file *101\_scaling\_approach.pdf*.

## B Appendix: Source code

The source code used in order to derive the IT artefact of this work is included on the attached disk. The source code includes:

- ***ubc-bondingcurve***: The basic Go implementation used for deriving the fitting algorithm in cycles one, two and three.
- ***ubc-python***: The Python implementation of the UBC, used for plotting UBC curves, testing of the actual UBC-MM Cosmos implementation and plotting via API calls.
- ***ubc-cosmos***: The final UBC-MM implementation using the Cosmos SDK. The entire blockchain can be started using the Ignite CLI.

# References

- Aflak, O. (2020, July 27). *Bézier curve* [Medium]. Retrieved May 27, 2022, from <https://towardsdatascience.com/b%C3%A9zier-curve-bfffdadea212>
- Angeris, G., & Chitra, T. (2020). Improved price oracles: Constant function market makers. *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, 80–91. <https://doi.org/10.1145/3419614.3423251>
- Balasanov, S. (2020, March 8). *Bonding curves in depth: Intuition & parametrization* [Medium]. Retrieved April 1, 2022, from <https://blog.relevant.community/bonding-curves-in-depth-intuition-parametrization-d3905a681e0a>
- BlockScience. (2022, March 11). *Alpha bonds: Risk-adjusted bonding curves for financing impact* [BlockScience]. Retrieved April 1, 2022, from <https://medium.com/block-science/alpha-bonds-risk-adjusted-bonding-curves-for-financing-impact-5be949fbc5a0>
- Bronstein, I., Semendyayev, K., Musiol, G., & Muehlig, H. (2007). *Handbook of mathematics* (5th ed). Springer.
- Buchman, E. (2016). Tendermint: Byzantine fault tolerance in the age of blockchains. *The University of Guelph*, 109. <http://hdl.handle.net/10214/9769>
- Chainalysis. (2022, June 6). *DeFi-driven speculation pushes decentralized exchanges' on-chain transaction volumes past centralized platforms* [Chainalysis]. Retrieved July 12, 2022, from <https://blog.chainalysis.com/reports/defi-dexs-web3/>
- Cosmos Network. (2022a). *Application-specific blockchains | cosmos SDK*. Retrieved May 14, 2022, from <https://docs.cosmos.network/master/intro/why-app-specific.html>
- Cosmos Network. (2022b). *A blockchain app architecture | cosmos developer portal*. Retrieved May 14, 2022, from <https://tutorials.cosmos.network/academy/2-main-concepts/architecture.html>
- Cosmos Network. (2022c). *Main components of the cosmos SDK | cosmos SDK*. Retrieved May 14, 2022, from <https://docs.cosmos.network/master/intro/sdk-design.html>
- Cosmos SDK. (2022a, July 6). *Cosmos SDK github - math/decimal.go* [original-date: 2016-02-06T07:15:53Z]. COSMOS. Retrieved July 6, 2022, from <https://github.com/cosmos/cosmos-sdk/blob/f70b67fc917baeb5060d63f124a9031ce1210c6a/types/decimal.go>

- Cosmos SDK. (2022b, July 6). *Cosmos SDK github - math/int.go* [original-date: 2016-02-06T07:15:53Z]. COSMOS. Retrieved July 6, 2022, from <https://github.com/cosmos/cosmos-sdk/blob/f70b67fc917baeb5060d63f124a9031ce1210c6a/math/int.go>
- Coutts, V. (2019, September 23). *An introduction to bonding curves, shapes and use cases* [Linum labs blog]. Retrieved July 12, 2022, from <https://medium.com/linum-labs/intro-to-bonding-curves-and-shapes-bf326bc4e11a>
- Edelman, G. (2021). The father of web3 wants you to trust less [Section: tags]. *Wired*. Retrieved July 11, 2022, from <https://www.wired.com/story/web3-gavin-wood-interview/>
- Evans, A. (2020, July 14). Liquidity provider returns in geometric mean markets [Number: arXiv:2006.08806]. Retrieved May 16, 2022, from <http://arxiv.org/abs/2006.08806>
- Farmer, J. D. (2000). Market force, ecology and evolution. *arXiv:adap-org/9812005*. Retrieved February 3, 2022, from <http://arxiv.org/abs/adap-org/9812005>
- Favre, T. (2018). *Introducing continuous organizations* | *HackerNoon*. Retrieved May 21, 2022, from <https://hackernoon.com/introducing-continuous-organizations-22ad9d1f63b7>
- Gaia-x. (2022). *GAIA-x home* [GAIA-x - home]. Retrieved July 12, 2022, from <https://www.data-infrastructure.eu/GAIA-X/Navigation/EN/Home/home.html>
- Ganser, O. (2021, November 25). *Catena-x: Eine dateninfrastruktur für die automobile wertschöpfungskette* | *#DMCO21*. Retrieved July 12, 2022, from <https://www.youtube.com/watch?v=Df4-qpLfw8E>
- Golang Docs. (2022a). *Source file src/math/big/int.go* [Source file src/math/big/int.go]. Retrieved July 6, 2022, from <https://go.dev/src/math/big/int.go>
- Golang Docs. (2022b). *Source file src/math/big/nat.go* [Source file src/math/big/nat.go]. Retrieved July 6, 2022, from <https://go.dev/src/math/big/nat.go>
- Gregor, S., & Hevner, A. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37, 337–356. <https://doi.org/10.25300/MISQ/2013/37.2.01>
- Han, X.-A., Ma, Y., & Huang, X. (2008). A novel generalization of bézier curve and surface. *Journal of Computational and Applied Mathematics*, 217(1), 180–193. <https://doi.org/10.1016/j.cam.2007.06.027>
- Hevner, A., R, A., March, S., T, S., Park, Park, J., Ram, & Sudha. (2004). Design science in information systems research. *Management Information Systems Quarterly*, 28, 75.

- Ignite Cli Docs. (2022a). *CLI reference / ignite cli docs* [Ignite CLI]. Retrieved July 5, 2022, from <https://docs.ignite.com/cli>
- Ignite Cli Docs. (2022b). *Ignite CLI / ignite cli docs* [Ignite CLI | ignite cli docs]. Retrieved July 27, 2022, from <https://docs.ignite.com/>
- Kaal, W. A., & Dell'Erba, M. (2017, November 8). Initial coin offerings: Emerging practices, risk factors, and red flags. <https://doi.org/10.2139/ssrn.3067615>
- Kharif, O. (2021). What you need to know about web3, crypto's attempt to reinvent the internet. *Bloomberg.com*. Retrieved July 11, 2022, from <https://www.bloomberg.com/news/articles/2021-12-10/web3-is-crypto-s-attempt-to-reinvent-the-internet-here-s-what-you-should-know>
- Kobylinska, A., Martin, F. P., & Schmitz, P. (2021, September 8). *Blockchains im operativen Geschäft* [Blockchain Insider]. Retrieved July 12, 2022, from <https://www.blockchain-insider.de/blockchains-im-operativen-geschaeft-a-1054034/>
- Kwon, J., & Buchmann, E. (2019). Cosmos: A network of distributed ledgers. Retrieved February 2, 2022, from <https://v1.cosmos.network/resources/whitepaper>
- Lamberty, R., & Poddey, A. (2020). Regulation conform DLT-operable payment adapter based on trustless - justified trust combined generalized state channels. *arXiv:2007.01605 [cs]*. Retrieved July 23, 2021, from <http://arxiv.org/abs/2007.01605>
- Lau, W. (2018, August 8). *On single bonding curves for continuous token models* [Thoughtchains]. Retrieved February 21, 2022, from <https://medium.com/thoughtchains/on-single-bonding-curves-for-continuous-token-models-a167f5ffef89>
- Li, T., Shin, D., & Wang, B. (2021, February 10). Cryptocurrency pump-and-dump schemes. <https://doi.org/10.2139/ssrn.3267041>
- Othman, A., Pennock, D. M., Reeves, D. M., & Sandholm, T. (2013). A practical liquidity-sensitive automated market maker. *ACM Transactions on Economics and Computation*, 1(3), 1–25. <https://doi.org/10.1145/2509413.2509414>
- Pease, M., Shostak, R., & Lamport, L. (1980). Reaching agreement in the presence of faults. *Journal of the ACM*, 27(2). <https://doi.org/10.1145/322186.322188>
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Poddey, A., & Lamberty, R. (2022). Coopetitive digital society & tokenization notebook. *Internal Document*.

- Poddey, A., & Scharmann, N. (2019, August 30). On the importance of system-view centric validation for the design and operation of a crypto-based digital economy. Retrieved July 11, 2022, from <http://arxiv.org/abs/1908.08675>
- Prautzsch, H., Boehm, W., & Paluszny, M. (2002). *Bézier and b-spline techniques*. Springer Berlin Heidelberg. <https://doi.org/10.1007/978-3-662-04919-8>
- Rouviere, S. d. l. (2019, August 28). *Tokens 2.0: Curved token bonding in curation markets* [Medium]. Retrieved July 11, 2022, from <https://medium.com/@simondlr/tokens-2-0-curved-token-bonding-in-curation-markets-1764a2e0bee5>
- Staking Rewards. (2022, May 17). *Biggest cryptocurrencies in the world based on total staked value* [Statista. statista inc.]. Retrieved July 12, 2022, from <https://www.statista.com/statistics/1279011/crypto-staked-value/>
- Titcomb, A. (2020, July 8). *Deep dive: Augmented bonding curves* [Commons stack]. Retrieved April 1, 2022, from <https://medium.com/commonsstack/deep-dive-augmented-bonding-curves-b5ca4fad4436>
- Voshmgir, S. (2019). *Token economy: How blockchains and smart contracts revolutionize the economy* (1st edition, 2nd amended printing). BlockchainHub.
- Weisstein, E. W. (2022). *Least squares fitting* [MathWorld] [Publisher: Wolfram Research, Inc.]. Retrieved July 4, 2022, from <https://mathworld.wolfram.com/LeastSquaresFitting.html>
- Wieringa, R. J. (2014). *Design science methodology for information systems and software engineering*. Springer Berlin Heidelberg. <https://doi.org/10.1007/978-3-662-43839-8>
- Xu, J., Vavryk, N., Paruch, K., & Cousaert, S. (2021). SoK: Decentralized exchanges (DEX) with automated market maker (AMM) protocols. *arXiv:2103.12732 [cs, q-fin]*. Retrieved July 23, 2021, from <http://arxiv.org/abs/2103.12732>
- Zargham, M., Shorish, J., & Paruch, K. (2020). From curved bonding to configuration spaces. *2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 1–3. <https://doi.org/10.1109/ICBC48266.2020.9169474>
- Zetzsche, D. A., Buckley, R. P., Arner, D. W., & FFhr, L. (2017). The ICO gold rush: It's a scam, it's a bubble, it's a super challenge for regulators. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.3072298>
- Zhu, M., Chiarella, C., He, X.-Z., & Wang, D. (2009). Does the market maker stabilize the market? *Physica A: Statistical Mechanics and its Applications*, 388(15), 3164–3180. <https://doi.org/10.1016/j.physa.2009.04.013>
- Zinsmeister, N., & Robinson, D. (2020). Uniswap v2 core, 10. <https://uniswap.org/whitepaper.pdf>